

Semana 11 - Redes Convolucionais Clássicas

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

Estudos de Caso

- Veremos exemplos de redes convolucionais que efetivamente funcionam
- Assim como estudar códigos de outras pessoas ajuda a evoluir na programação, estudar esses modelos ajuda a ganhar intuição de como montar as peças do modelo
- E arquiteturas que funcionam bem em outros problemas tendem a funcionar bem no seu também
- Aqui veremos:
 - Redes Clássicas: LeNet-5, AlexNet, VGG
 - ResNet (152)
 - Inception

Estudos de Caso

- Veremos exemplos de redes convolucionais que efetivamente funcionam
- Assim como estudar códigos de outras pessoas ajuda a evoluir na programação, estudar esses modelos ajuda a ganhar intuição de como montar as peças do modelo
- E arquiteturas que funcionam bem em outros problemas tendem a funcionar bem no seu também
- Aqui veremos:
 - Redes Clássicas: LeNet-5, AlexNet, VGG
 - ResNet (152)
 - Inception

Estudos de Caso

- Veremos exemplos de redes convolucionais que efetivamente funcionam
- Assim como estudar códigos de outras pessoas ajuda a evoluir na programação, estudar esses modelos ajuda a ganhar intuição de como montar as peças do modelo
- E arquiteturas que funcionam bem em outros problemas tendem a funcionar bem no seu também
- Aqui veremos:
 - Redes Clássicas: LeNet-5, AlexNet, VGG
 - ResNet (152)
 - Inception

Estudos de Caso

- Veremos exemplos de redes convolucionais que efetivamente funcionam
- Assim como estudar códigos de outras pessoas ajuda a evoluir na programação, estudar esses modelos ajuda a ganhar intuição de como montar as peças do modelo
- E arquiteturas que funcionam bem em outros problemas tendem a funcionar bem no seu também
- Aqui veremos:
 - Redes Clássicas: LeNet-5, AlexNet, VGG
 - ResNet (152)
 - Inception

Estudos de Caso

- Veremos exemplos de redes convolucionais que efetivamente funcionam
- Assim como estudar códigos de outras pessoas ajuda a evoluir na programação, estudar esses modelos ajuda a ganhar intuição de como montar as peças do modelo
- E arquiteturas que funcionam bem em outros problemas tendem a funcionar bem no seu também
- Aqui veremos:
 - Redes Clássicas: LeNet-5, AlexNet, VGG
 - ResNet (152)
 - Inception

Estudos de Caso

- Veremos exemplos de redes convolucionais que efetivamente funcionam
- Assim como estudar códigos de outras pessoas ajuda a evoluir na programação, estudar esses modelos ajuda a ganhar intuição de como montar as peças do modelo
- E arquiteturas que funcionam bem em outros problemas tendem a funcionar bem no seu também
- Aqui veremos:
 - Redes Clássicas: LeNet-5, AlexNet, VGG
 - ResNet (152)
 - Inception

Estudos de Caso

- Veremos exemplos de redes convolucionais que efetivamente funcionam
- Assim como estudar códigos de outras pessoas ajuda a evoluir na programação, estudar esses modelos ajuda a ganhar intuição de como montar as peças do modelo
- E arquiteturas que funcionam bem em outros problemas tendem a funcionar bem no seu também
- Aqui veremos:
 - Redes Clássicas: LeNet-5, AlexNet, VGG
 - ResNet (152)
 - Inception

Estudos de Caso

- NOTA 1: A ResNet é uma rede muito profunda (152 camadas) e veremos os truques usados para treiná-la
- NOTA 2: Muitas das ideias em arquiteturas como a ResNet e a Inception se mostraram também úteis em outros domínios fora da visão computacional

Estudos de Caso

- NOTA 1: A ResNet é uma rede muito profunda (152 camadas) e veremos os truques usados para treiná-la
- NOTA 2: Muitas das ideias em arquiteturas como a ResNet e a Inception se mostraram também úteis em outros domínios fora da visão computacional

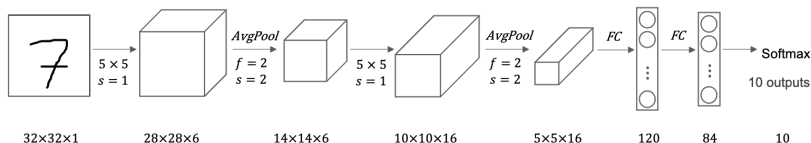
Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas**
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

LeNet-5¹

- Seja um problema de reconhecimento de dígitos em imagens 32x32 em tons de cinza (um canal)

LeNet-5



¹LeCun et al., 1998. Gradient-based learning applied to document recognition

LeNet-5 Notas

- O modelo original usava *average pooling* porque era mais popular na época - 1998 (hoje usaria *max pooling*)
- O modelo original usava um classificador diferente na saída, mas atualmente a *softmax* é muito mais usada
- Rede pequena para os moldes atuais, com 60K parâmetros, hoje é comum vermos redes com 10 ou 100M parâmetros
- À medida que vamos mais fundo nas camadas, a altura e largura diminuem e o número de canais aumenta (padrão recorrente em CNNs)
- Outro padrão, presente ainda hoje, é a sequência conv-pool-conv-pool-fc-fc-saída

LeNet-5 Notas

- O modelo original usava *average pooling* porque era mais popular na época - 1998 (hoje usaria *max pooling*)
- O modelo original usava um classificador diferente na saída, mas atualmente a *softmax* é muito mais usada
- Rede pequena para os moldes atuais, com 60K parâmetros, hoje é comum vermos redes com 10 ou 100M parâmetros
- À medida que vamos mais fundo nas camadas, a altura e largura diminuem e o número de canais aumenta (padrão recorrente em CNNs)
- Outro padrão, presente ainda hoje, é a sequência conv-pool-conv-pool-fc-fc-saída

LeNet-5 Notas

- O modelo original usava *average pooling* porque era mais popular na época - 1998 (hoje usaria *max pooling*)
- O modelo original usava um classificador diferente na saída, mas atualmente a *softmax* é muito mais usada
- Rede pequena para os moldes atuais, com 60K parâmetros, hoje é comum vermos redes com 10 ou 100M parâmetros
- À medida que vamos mais fundo nas camadas, a altura e largura diminuem e o número de canais aumenta (padrão recorrente em CNNs)
- Outro padrão, presente ainda hoje, é a sequência conv-pool-conv-pool-fc-fc-saída

LeNet-5 Notas

- O modelo original usava *average pooling* porque era mais popular na época - 1998 (hoje usaria *max pooling*)
- O modelo original usava um classificador diferente na saída, mas atualmente a *softmax* é muito mais usada
- Rede pequena para os moldes atuais, com 60K parâmetros, hoje é comum vermos redes com 10 ou 100M parâmetros
- À medida que vamos mais fundo nas camadas, a altura e largura diminuem e o número de canais aumenta (padrão recorrente em CNNs)
- Outro padrão, presente ainda hoje, é a sequência conv-pool-conv-pool-fc-fc-saída

LeNet-5 Notas

- O modelo original usava *average pooling* porque era mais popular na época - 1998 (hoje usaria *max pooling*)
- O modelo original usava um classificador diferente na saída, mas atualmente a *softmax* é muito mais usada
- Rede pequena para os moldes atuais, com 60K parâmetros, hoje é comum vermos redes com 10 ou 100M parâmetros
- À medida que vamos mais fundo nas camadas, a altura e largura diminuem e o número de canais aumenta (padrão recorrente em CNNs)
- Outro padrão, presente ainda hoje, é a sequência conv-pool-conv-pool-fc-fc-saída

LeNet-5 Notas

- Na época, usava-se sigmoide/tanh como não-linearidade; hoje se usaria ReLU
- Devido aos computadores bem mais lentos da época, a convolução 3D não era como vimos; havia uma estratégia complicada com diferentes filtros olhando para diferentes canais
- Uma última coisa que não se usa mais é que havia uma não-linearidade (tanh) após a camada de *pooling*
- NOTA: A maior parte dessa descrição está nas Seções II e III do artigo; as seções após isso falam de outras ideias, como as *graph transformer networks*, que não são mais usadas

LeNet-5 Notas

- Na época, usava-se sigmoide/tanh como não-linearidade; hoje se usaria ReLU
- Devido aos computadores bem mais lentos da época, a convolução 3D não era como vimos; havia uma estratégia complicada com diferentes filtros olhando para diferentes canais
- Uma última coisa que não se usa mais é que havia uma não-linearidade (tanh) após a camada de *pooling*
- NOTA: A maior parte dessa descrição está nas Seções II e III do artigo; as seções após isso falam de outras ideias, como as *graph transformer networks*, que não são mais usadas

LeNet-5 Notas

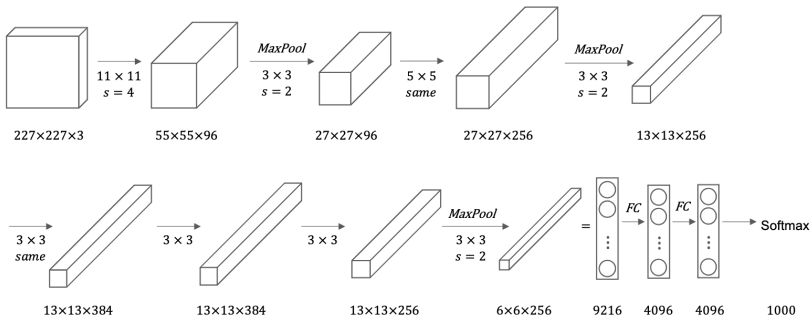
- Na época, usava-se sigmoide/tanh como não-linearidade; hoje se usaria ReLU
- Devido aos computadores bem mais lentos da época, a convolução 3D não era como vimos; havia uma estratégia complicada com diferentes filtros olhando para diferentes canais
- Uma última coisa que não se usa mais é que havia uma não-linearidade (tanh) após a camada de *pooling*
- NOTA: A maior parte dessa descrição está nas Seções II e III do artigo; as seções após isso falam de outras ideias, como as *graph transformer networks*, que não são mais usadas

LeNet-5 Notas

- Na época, usava-se sigmoide/tanh como não-linearidade; hoje se usaria ReLU
- Devido aos computadores bem mais lentos da época, a convolução 3D não era como vimos; havia uma estratégia complicada com diferentes filtros olhando para diferentes canais
- Uma última coisa que não se usa mais é que havia uma não-linearidade (tanh) após a camada de *pooling*
- NOTA: A maior parte dessa descrição está nas Seções II e III do artigo; as seções após isso falam de outras ideias, como as *graph transformer networks*, que não são mais usadas

AlexNet²

AlexNet



²Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks

AlexNet Notas

- NOTA: Todas as convoluções que mantêm o mesmo tamanho do tensor são do tipo *same*
- Tem semelhanças com a LeNet, mas é muito maior; enquanto a LeNet tinha 60K parâmetros, a AlexNet tem 60M
- Isso foi basicamente o que permitiu que ela tivesse um desempenho tão extraordinário!
- Outro avanço em relação à LeNet era o uso de ReLU e o treinamento em múltiplas GPUs
- Porém, esse uso de GPU era bem primitivo e mais complicado na época, sendo que a camada convolucional era dividida entre 2 GPUs e tinham que se comunicar

AlexNet Notas

- NOTA: Todas as convoluções que mantêm o mesmo tamanho do tensor são do tipo *same*
- Tem semelhanças com a LeNet, mas é muito maior; enquanto a LeNet tinha 60K parâmetros, a AlexNet tem 60M
- Isso foi basicamente o que permitiu que ela tivesse um desempenho tão extraordinário!
- Outro avanço em relação à LeNet era o uso de ReLU e o treinamento em múltiplas GPUs
- Porém, esse uso de GPU era bem primitivo e mais complicado na época, sendo que a camada convolucional era dividida entre 2 GPUs e tinham que se comunicar

AlexNet Notas

- NOTA: Todas as convoluções que mantêm o mesmo tamanho do tensor são do tipo *same*
- Tem semelhanças com a LeNet, mas é muito maior; enquanto a LeNet tinha 60K parâmetros, a AlexNet tem 60M
- Isso foi basicamente o que permitiu que ela tivesse um desempenho tão extraordinário!
- Outro avanço em relação à LeNet era o uso de ReLU e o treinamento em múltiplas GPUs
- Porém, esse uso de GPU era bem primitivo e mais complicado na época, sendo que a camada convolucional era dividida entre 2 GPUs e tinham que se comunicar

AlexNet Notas

- NOTA: Todas as convoluções que mantêm o mesmo tamanho do tensor são do tipo *same*
- Tem semelhanças com a LeNet, mas é muito maior; enquanto a LeNet tinha 60K parâmetros, a AlexNet tem 60M
- Isso foi basicamente o que permitiu que ela tivesse um desempenho tão extraordinário!
- Outro avanço em relação à LeNet era o uso de ReLU e o treinamento em múltiplas GPUs
- Porém, esse uso de GPU era bem primitivo e mais complicado na época, sendo que a camada convolucional era dividida entre 2 GPUs e tinham que se comunicar

AlexNet Notas

- NOTA: Todas as convoluções que mantêm o mesmo tamanho do tensor são do tipo *same*
- Tem semelhanças com a LeNet, mas é muito maior; enquanto a LeNet tinha 60K parâmetros, a AlexNet tem 60M
- Isso foi basicamente o que permitiu que ela tivesse um desempenho tão extraordinário!
- Outro avanço em relação à LeNet era o uso de ReLU e o treinamento em múltiplas GPUs
- Porém, esse uso de GPU era bem primitivo e mais complicado na época, sendo que a camada convolucional era dividida entre 2 GPUs e tinham que se comunicar

AlexNet Notas

- Outro conceito inovador era a *Local Response Normalization* (LRN): normalizar os valores em cada posição de altura e largura no bloco sobre todos os canais
- Assim evitava neurônios com ativação muito alta em cada posição; mas isso caiu em desuso!
- OBSERVAÇÃO: Embora já houvesse um crescimento do *deep learning* em áreas como reconhecimento de áudio e outras, foi este artigo que fez com que as pessoas olhassem de forma mais séria para essas ferramentas
- NOTA: Para quem for ler, este é um dos artigos mais fáceis do *deep learning*

AlexNet Notas

- Outro conceito inovador era a *Local Response Normalization* (LRN): normalizar os valores em cada posição de altura e largura no bloco sobre todos os canais
- Assim evitava neurônios com ativação muito alta em cada posição; mas isso caiu em desuso!
- OBSERVAÇÃO: Embora já houvesse um crescimento do *deep learning* em áreas como reconhecimento de áudio e outras, foi este artigo que fez com que as pessoas olhassem de forma mais séria para essas ferramentas
- NOTA: Para quem for ler, este é um dos artigos mais fáceis do *deep learning*

AlexNet Notas

- Outro conceito inovador era a *Local Response Normalization* (LRN): normalizar os valores em cada posição de altura e largura no bloco sobre todos os canais
- Assim evitava neurônios com ativação muito alta em cada posição; mas isso caiu em desuso!
- OBSERVAÇÃO: Embora já houvesse um crescimento do *deep learning* em áreas como reconhecimento de áudio e outras, foi este artigo que fez com que as pessoas olhassem de forma mais séria para essas ferramentas
- NOTA: Para quem for ler, este é um dos artigos mais fáceis do *deep learning*

AlexNet Notas

- Outro conceito inovador era a *Local Response Normalization* (LRN): normalizar os valores em cada posição de altura e largura no bloco sobre todos os canais
- Assim evitava neurônios com ativação muito alta em cada posição; mas isso caiu em desuso!
- OBSERVAÇÃO: Embora já houvesse um crescimento do *deep learning* em áreas como reconhecimento de áudio e outras, foi este artigo que fez com que as pessoas olhassem de forma mais séria para essas ferramentas
- NOTA: Para quem for ler, este é um dos artigos mais fáceis do *deep learning*

AlexNet Notas

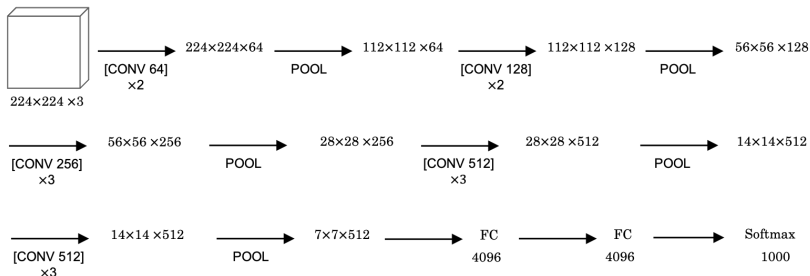
- CURIOSIDADE 1: Além do Alex Krizhevsky, os outros autores são Ilya Sutskever e Geoffrey Hinton
- CURIOSIDADE 2: O artigo original fala de uma entrada 224×224 , mas os números só fazem sentido para 227×227

AlexNet Notas

- CURIOSIDADE 1: Além do Alex Krizhevsky, os outros autores são Ilya Sutskever e Geoffrey Hinton
- CURIOSIDADE 2: O artigo original fala de uma entrada 224×224 , mas os números só fazem sentido para 227×227

VGG-16³

VGG-16

CONV = 3×3 filter, $s = 1$, sameMAX-POOL = 2×2 , $s = 2$ 

³Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition

VGG-16 Notas

- O destaque aqui é a simplificação da arquitetura o número bem menor hiperparâmetros pois se usa sempre o mesmo tipo de operação: conv 3×3 , $s=1$, *same* e max-pool 2×2 , $s=2$
- NOTA: Conv64 refere-se a 64 filtros 3×3 de convolução no formato padrão acima
- CURIOSIDADE: O número 16 em VGG-16 vem de haver 16 camadas com pesos
- Trata-se de uma rede muito grande (ainda hoje) com 138M parâmetros!
- Porém a simplicidade e uniformidade dessa arquitetura fazem com que ela seja interessante

VGG-16 Notas

- O destaque aqui é a simplificação da arquitetura o número bem menor hiperparâmetros pois se usa sempre o mesmo tipo de operação: conv 3×3 , $s=1$, *same* e max-pool 2×2 , $s=2$
- NOTA: Conv64 refere-se a 64 filtros 3×3 de convolução no formato padrão acima
- CURIOSIDADE: O número 16 em VGG-16 vem de haver 16 camadas com pesos
- Trata-se de uma rede muito grande (ainda hoje) com 138M parâmetros!
- Porém a simplicidade e uniformidade dessa arquitetura fazem com que ela seja interessante

VGG-16 Notas

- O destaque aqui é a simplificação da arquitetura o número bem menor hiperparâmetros pois se usa sempre o mesmo tipo de operação: conv 3×3 , $s=1$, *same* e max-pool 2×2 , $s=2$
- NOTA: Conv64 refere-se a 64 filtros 3×3 de convolução no formato padrão acima
- CURIOSIDADE: O número 16 em VGG-16 vem de haver 16 camadas com pesos
- Trata-se de uma rede muito grande (ainda hoje) com 138M parâmetros!
- Porém a simplicidade e uniformidade dessa arquitetura fazem com que ela seja interessante

VGG-16 Notas

- O destaque aqui é a simplificação da arquitetura o número bem menor hiperparâmetros pois se usa sempre o mesmo tipo de operação: conv 3×3 , $s=1$, *same* e max-pool 2×2 , $s=2$
- NOTA: Conv64 refere-se a 64 filtros 3×3 de convolução no formato padrão acima
- CURIOSIDADE: O número 16 em VGG-16 vem de haver 16 camadas com pesos
- Trata-se de uma rede muito grande (ainda hoje) com 138M parâmetros!
- Porém a simplicidade e uniformidade dessa arquitetura fazem com que ela seja interessante

VGG-16 Notas

- O destaque aqui é a simplificação da arquitetura o número bem menor hiperparâmetros pois se usa sempre o mesmo tipo de operação: conv 3×3 , $s=1$, *same* e max-pool 2×2 , $s=2$
- NOTA: Conv64 refere-se a 64 filtros 3×3 de convolução no formato padrão acima
- CURIOSIDADE: O número 16 em VGG-16 vem de haver 16 camadas com pesos
- Trata-se de uma rede muito grande (ainda hoje) com 138M parâmetros!
- Porém a simplicidade e uniformidade dessa arquitetura fazem com que ela seja interessante

VGG-16 Notas

- NOTA: Existe na literatura uma versão ainda maior chamada VGG-19, embora normalmente a 16 funcione tão bem quanto a 19
- Note novamente o padrão de diminuir altura/largura e aumentar canais em cada camada
- NOTA: Para quem for ler estes artigos, a ordem sugerida é AlexNet → VGG → LeNet

VGG-16 Notas

- NOTA: Existe na literatura uma versão ainda maior chamada VGG-19, embora normalmente a 16 funcione tão bem quanto a 19
- Note novamente o padrão de diminuir altura/largura e aumentar canais em cada camada
- NOTA: Para quem for ler estes artigos, a ordem sugerida é AlexNet → VGG → LeNet

VGG-16 Notas

- NOTA: Existe na literatura uma versão ainda maior chamada VGG-19, embora normalmente a 16 funcione tão bem quanto a 19
- Note novamente o padrão de diminuir altura/largura e aumentar canais em cada camada
- NOTA: Para quem for ler estes artigos, a ordem sugerida é AlexNet → VGG → LeNet

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet**
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

ResNet⁴

- Redes muito profundas são difíceis de treinar devido ao gradiente desaparecendo ou explodindo
- Usar *skip connections*: ativação de uma camada alimenta outra camada muito mais profunda na rede
- Isso permite o treino de redes muito profundas (mais de 100 camadas)
- O elemento básico é o chamado **bloco residual**

⁴He et al., 2015. Deep residual networks for image recognition.

ResNet⁴

- Redes muito profundas são difíceis de treinar devido ao gradiente desaparecendo ou explodindo
- Usar *skip connections*: ativação de uma camada alimenta outra camada muito mais profunda na rede
- Isso permite o treino de redes muito profundas (mais de 100 camadas)
- O elemento básico é o chamado **bloco residual**

⁴He et al., 2015. Deep residual networks for image recognition.

ResNet⁴

- Redes muito profundas são difíceis de treinar devido ao gradiente desaparecendo ou explodindo
- Usar *skip connections*: ativação de uma camada alimenta outra camada muito mais profunda na rede
- Isso permite o treino de redes muito profundas (mais de 100 camadas)
- O elemento básico é o chamado **bloco residual**

⁴He et al., 2015. Deep residual networks for image recognition.

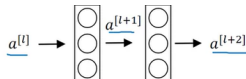
ResNet⁴

- Redes muito profundas são difíceis de treinar devido ao gradiente desaparecendo ou explodindo
- Usar *skip connections*: ativação de uma camada alimenta outra camada muito mais profunda na rede
- Isso permite o treino de redes muito profundas (mais de 100 camadas)
- O elemento básico é o chamado **bloco residual**

⁴He et al., 2015. Deep residual networks for image recognition.

ResNet

- Fluxo de uma rede típica:



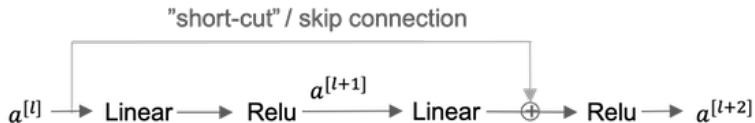
$$\underline{z^{[l+1]} = W^{[l+1]} \underline{a^{[l]} + b^{[l+1]}} \quad \underline{a^{[l+1]} = g(z^{[l+1]})} \quad \underline{z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}} \quad \underline{a^{[l+2]} = g(z^{[l+2]})}$$

Este fluxo de $a^{[l]}$ para $a^{[l+2]}$ forma o **caminho principal (main path)**

ResNet

- No bloco residual temos o **atalho** (**short cut** ou **skip connection**): $a^{[l]}$ é copiado para uma posição bem à frente na rede e somado com a saída naquele ponto

Residual block



$$\begin{aligned}
 z^{[l+1]} &= W^{[l+1]} a^{[l]} + b^{[l+1]} \\
 a^{[l+1]} &= g(z^{[l+1]}) \\
 z^{[l+2]} &= W^{[l+2]} a^{[l+1]} + b^{[l+2]} \\
 a^{[l+2]} &= g(z^{[l+2]})
 \end{aligned}$$



$$\begin{aligned}
 z^{[l+1]} &= W^{[l+1]} a^{[l]} + b^{[l+1]} \\
 a^{[l+1]} &= g(z^{[l+1]}) \\
 z^{[l+2]} &= W^{[l+2]} a^{[l+1]} + b^{[l+2]} \\
 a^{[l+2]} &= g(z^{[l+2]} + a^{[l]})
 \end{aligned}$$

ResNet

- NOTA: A entrada do dado pelo *short cut* se dá após uma soma linear e antes de uma ReLU
- A ResNet é formada empilhando-se vários desses blocos residuais
- Partindo de uma rede típica (chamada “*plain network*” no artigo), adiciona-se um salto a cada 2 camadas; Exemplo com 5 blocos residuais:

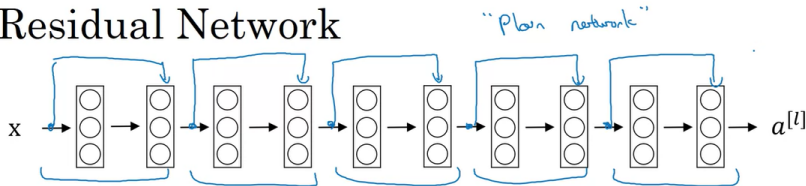
ResNet

- NOTA: A entrada do dado pelo *short cut* se dá após uma soma linear e antes de uma ReLU
- A ResNet é formada empilhando-se vários desses blocos residuais
- Partindo de uma rede típica (chamada “*plain network*” no artigo), adiciona-se um salto a cada 2 camadas; Exemplo com 5 blocos residuais:

ResNet

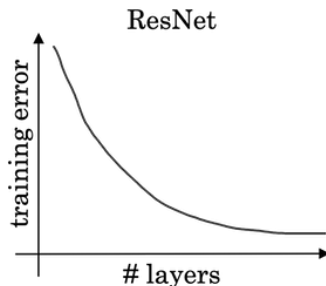
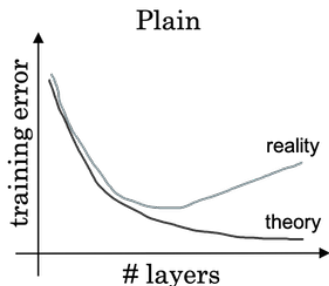
- NOTA: A entrada do dado pelo *short cut* se dá após uma soma linear e antes de uma ReLU
- A ResNet é formada empilhando-se vários desses blocos residuais
- Partindo de uma rede típica (chamada "*plain network*" no artigo), adiciona-se um salto a cada 2 camadas; Exemplo com 5 blocos residuais:

Residual Network



ResNet e as Redes Profundas

- Na teoria, o erro de treino deveria sempre diminuir quando o número de camadas em uma rede típica aumenta
- Mas na prática não é bem isso que ocorre
- A ResNet, porém, permite que o erro de treino continue diminuindo



ResNet - Por que Funcionam?

- Ir bem no treino é pré-requisito para um bom desempenho no teste
- Usando regularização (L2 ou *weight decay*), temos:

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]}) = g(W^{[l+2]}a^{[l+1]} + b^{[l+2]} + a^{[l]})$$

Se $W^{[l+2]} \rightarrow 0$ e $b^{[l+2]} \rightarrow 0$, temos, usando ReLU,

$$a^{[l+2]} = g(a^{[l]}) = a^{[l]}.$$

- Nesse caso o bloco residual aprende facilmente a função identidade

ResNet - Por que Funcionam?

- Ir bem no treino é pré-requisito para um bom desempenho no teste
- Usando regularização (L2 ou *weight decay*), temos:

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]}) = g(W^{[l+2]}a^{[l+1]} + b^{[l+2]} + a^{[l]})$$

Se $W^{[l+2]} \rightarrow 0$ e $b^{[l+2]} \rightarrow 0$, temos, usando ReLU,

$$a^{[l+2]} = g(a^{[l]}) = a^{[l]}.$$

- Nesse caso o bloco residual aprende facilmente a função identidade

ResNet - Por que Funcionam?

- Ir bem no treino é pré-requisito para um bom desempenho no teste
- Usando regularização (L2 ou *weight decay*), temos:

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]}) = g(W^{[l+2]}a^{[l+1]} + b^{[l+2]} + a^{[l]})$$

Se $W^{[l+2]} \rightarrow 0$ e $b^{[l+2]} \rightarrow 0$, temos, usando ReLU,

$$a^{[l+2]} = g(a^{[l]}) = a^{[l]}.$$

- Nesse caso o bloco residual aprende facilmente a função identidade

ResNet - Por que Funcionam?

- A rede funcionaria então como a rede menor, sem as 2 camadas extras
- Mas isso sem impedir também que a rede melhore usando o aprendizado dessas camadas extras
- Isso não ocorre em uma *plain network* muito profunda, em que mesmo a função identidade se torna difícil de aprender

ResNet - Por que Funcionam?

- A rede funcionaria então como a rede menor, sem as 2 camadas extras
- Mas isso sem impedir também que a rede melhore usando o aprendizado dessas camadas extras
- Isso não ocorre em uma *plain network* muito profunda, em que mesmo a função identidade se torna difícil de aprender

ResNet - Por que Funcionam?

- A rede funcionaria então como a rede menor, sem as 2 camadas extras
- Mas isso sem impedir também que a rede melhore usando o aprendizado dessas camadas extras
- Isso não ocorre em uma *plain network* muito profunda, em que mesmo a função identidade se torna difícil de aprender

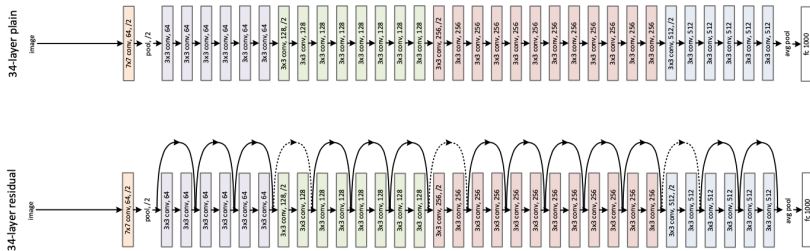
ResNet - Dimensões

- Na ResNet assumimos que $z^{[l+2]}$ e $a^{[l]}$ têm a mesma dimensão (usar convoluções *same*)
- Se tiverem dimensões diferentes, p.ex., $a^{[l]}$ tem 256 e $a^{[l+2]}$ tem 128, deve-se multiplicar $a^{[l]}$ por uma matriz W_s 256x128, cujos valores podem ser aprendidos ou fixos (usando *zero padding*)

ResNet - Dimensões

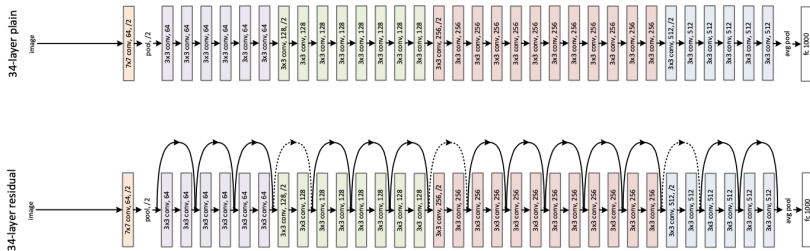
- Na ResNet assumimos que $z^{[l+2]}$ e $a^{[l]}$ têm a mesma dimensão (usar convoluções *same*)
- Se tiverem dimensões diferentes, p.ex., $a^{[l]}$ tem 256 e $a^{[l+2]}$ tem 128, deve-se multiplicar $a^{[l]}$ por uma matriz W_s 256x128, cujos valores podem ser aprendidos ou fixos (usando *zero padding*)

ResNet - Architettura Original



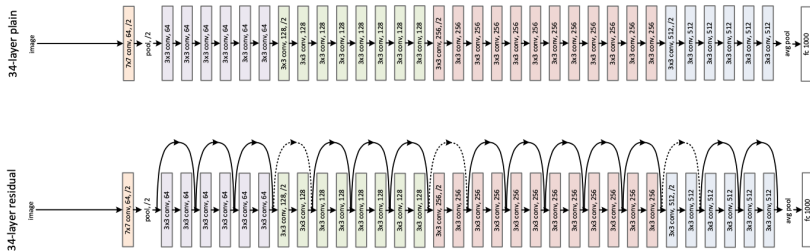
- Note que a maioria das conv 3x3 são *same*
- Como temos *pooling*, que altera as dimensões, sempre que necessário se usa a matriz W_s que vimos
- CURIOSIDADE: Os autores são Kaiming He, Xiangyu Zhang, Shaoqing Ren e Jian Sun

ResNet - Arquitetura Original



- Note que a maioria das conv 3x3 são *same*
- Como temos *pooling*, que altera as dimensões, sempre que necessário se usa a matriz W_s que vimos
- CURIOSIDADE: Os autores são Kaiming He, Xiangyu Zhang, Shaoqing Ren e Jian Sun

ResNet - Arquitetura Original



- Note que a maioria das conv 3x3 são *same*
- Como temos *pooling*, que altera as dimensões, sempre que necessário se usa a matriz W_s que vimos
- CURIOSIDADE: Os autores são Kaiming He, Xiangyu Zhang, Shaoqing Ren e Jian Sun

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1**
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

Convolução 1x1

- O que é isso? Multiplicar por um escalar?
- Suponha uma imagem 6x6 (6x6x1); neste caso, uma convolução 1x1 não faria nada interessante
- Já se temos, p.ex., 32 canais, aí faz muito mais sentido: cada posição (altura/largura) da imagem vai ser multiplicada por um filtro 1x1x32
- O produto ponto-a-ponto é então somado, aplicado à ReLU e colocado na posição da saída

Convolução 1x1

- O que é isso? Multiplicar por um escalar?
- Suponha uma imagem 6x6 (6x6x1); neste caso, uma convolução 1x1 não faria nada interessante

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6 × 6 × 1

*

2



=

2	4	6	...		

- Já se temos, p.ex., 32 canais, aí faz muito mais sentido: cada posição (altura/largura) da imagem vai ser multiplicada por um filtro 1x1x32
- O produto ponto-a-ponto é então somado, aplicado à ReLU e colocado na posição da saída

Convolução 1x1

- O que é isso? Multiplicar por um escalar?
- Suponha uma imagem 6x6 (6x6x1); neste caso, uma convolução 1x1 não faria nada interessante

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6 x 6 × 1

*

2



=

2	4	6	...		

- Já se temos, p.ex., 32 canais, aí faz muito mais sentido: cada posição (altura/largura) da imagem vai ser multiplicada por um filtro 1x1x32
- O produto ponto-a-ponto é então somado, aplicado à ReLU e colocado na posição da saída

Convolução 1x1

- O que é isso? Multiplicar por um escalar?
- Suponha uma imagem 6x6 (6x6x1); neste caso, uma convolução 1x1 não faria nada interessante

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6 × 6 × 1

*

2

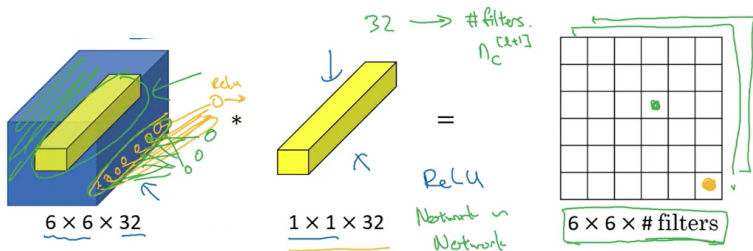


=

2	4	6	...		

- Já se temos, p.ex., 32 canais, aí faz muito mais sentido: cada posição (altura/largura) da imagem vai ser multiplicada por um filtro 1x1x32
- O produto ponto-a-ponto é então somado, aplicado à ReLU e colocado na posição da saída

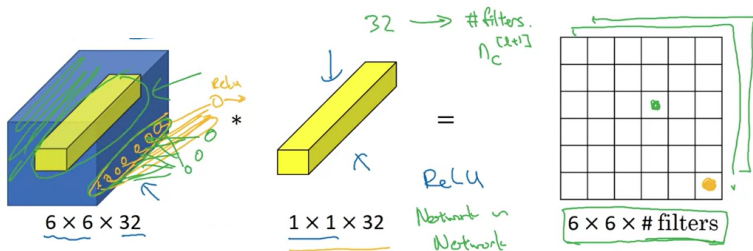
Convolução 1x1



- Equivale a uma rede totalmente conectada aplicada em cada uma dessas 36 posições
- E podemos ter várias redes dessas no caso de múltiplos filtros
- Essa estratégia é chamada também de *Network-in-Network*⁵

⁵Lin et al., 2013. Network in network.

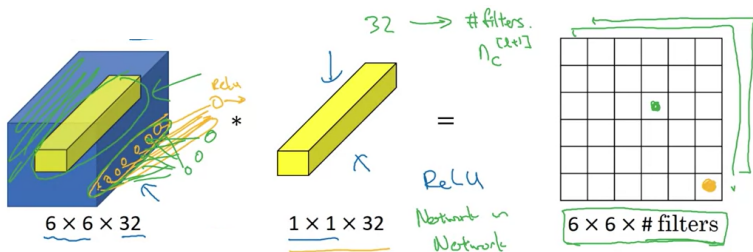
Convolução 1x1



- Equivale a uma rede totalmente conectada aplicada em cada uma dessas 36 posições
- E podemos ter várias redes dessas no caso de múltiplos filtros
- Essa estratégia é chamada também de *Network-in-Network*⁵

⁵Lin et al., 2013. Network in network.

Convolução 1x1



- Equivale a uma rede totalmente conectada aplicada em cada uma dessas 36 posições
- E podemos ter várias redes dessas no caso de múltiplos filtros
- Essa estratégia é chamada também de *Network-in-Network*⁵

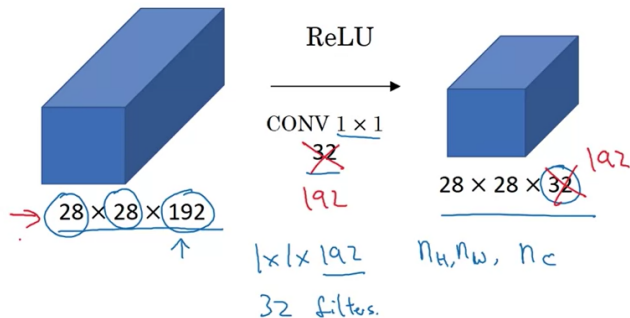
⁵Lin et al., 2013. Network in network.

Exemplo de Uso

- Para diminuir a altura/largura, usamos *pooling*
- Mas e para reduzir o número de canais, p.ex., de 192 para 32? Simplesmente usamos 32 filtros 1x1 (a rigor, 1x1x192)

Exemplo de Uso

- Para diminuir a altura/largura, usamos *pooling*
- Mas e para reduzir o número de canais, p.ex., de 192 para 32? Simplesmente usamos 32 filtros 1x1 (a rigor, 1x1x192)



Exemplo de Uso

- Isso permite reduzir a computação em algumas redes
- Outro truque é manter o número de canais (usando filtros 1x1x192) pois isso adiciona uma não-linearidade, aprendendo funções mais complexas
- O conceito será usado na rede Inception, que veremos em breve
- CURIOSIDADE: Os autores são M. Lin, Q. Chen, S. Yan

Exemplo de Uso

- Isso permite reduzir a computação em algumas redes
- Outro truque é manter o número de canais (usando filtros 1x1x192) pois isso adiciona uma não-linearidade, aprendendo funções mais complexas
- O conceito será usado na rede Inception, que veremos em breve
- CURIOSIDADE: Os autores são M. Lin, Q. Chen, S. Yan

Exemplo de Uso

- Isso permite reduzir a computação em algumas redes
- Outro truque é manter o número de canais (usando filtros $1 \times 1 \times 192$) pois isso adiciona uma não-linearidade, aprendendo funções mais complexas
- O conceito será usado na rede Inception, que veremos em breve
- CURIOSIDADE: Os autores são M. Lin, Q. Chen, S. Yan

Exemplo de Uso

- Isso permite reduzir a computação em algumas redes
- Outro truque é manter o número de canais (usando filtros $1 \times 1 \times 192$) pois isso adiciona uma não-linearidade, aprendendo funções mais complexas
- O conceito será usado na rede Inception, que veremos em breve
- CURIOSIDADE: Os autores são M. Lin, Q. Chen, S. Yan

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception**
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

Motivação da Rede Inception⁶

- Suponha um bloco $28 \times 28 \times 192$
- Nas redes até agora, escolhemos o tamanho do filtro e se queremos uma camada de convolução ou *pooling*
- O que uma camada de *inception* faz é tudo isso de uma vez

⁶Szegedy et al. 2014. Going deeper with convolutions.

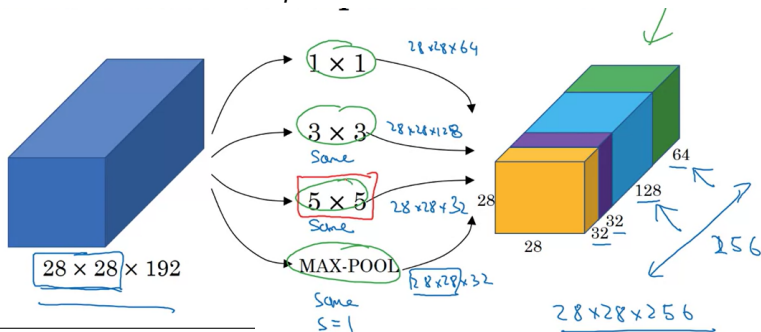
Motivação da Rede Inception⁶

- Suponha um bloco $28 \times 28 \times 192$
- Nas redes até agora, escolhemos o tamanho do filtro e se queremos uma camada de convolução ou *pooling*
- O que uma camada de *inception* faz é tudo isso de uma vez

⁶Szegedy et al. 2014. Going deeper with convolutions.

Motivação da Rede Inception⁶

- Suponha um bloco $28 \times 28 \times 192$
- Nas redes até agora, escolhemos o tamanho do filtro e se queremos uma camada de convolução ou *pooling*
- O que uma camada de *inception* faz é tudo isso de uma vez



⁶Szegedy et al. 2014. Going deeper with convolutions.

Motivação

- Aplicamos, p.ex., uma conv 1x1, outra 3x3, outra 5x5, e ainda um *max-pooling*
- Então empilhamos todas as saídas em um bloco só
- As conv 3x3 e 5x5 são do tipo *same* para preservar a dimensão e permitir o empilhamento
- Pelo mesmo motivo, usa-se *padding* no *max-pool* (do tipo *same*) e *stride s = 1*
- Este é o **módulo inception**; Note que sua saída é 28x28x256 no caso

Motivação

- Aplicamos, p.ex., uma conv 1×1 , outra 3×3 , outra 5×5 , e ainda um *max-pooling*
- Então empilhamos todas as saídas em um bloco só
- As conv 3×3 e 5×5 são do tipo *same* para preservar a dimensão e permitir o empilhamento
- Pelo mesmo motivo, usa-se *padding* no *max-pool* (do tipo *same*) e *stride* $s = 1$
- Este é o **módulo inception**; Note que sua saída é $28 \times 28 \times 256$ no caso

Motivação

- Aplicamos, p.ex., uma conv 1×1 , outra 3×3 , outra 5×5 , e ainda um *max-pooling*
- Então empilhamos todas as saídas em um bloco só
- As conv 3×3 e 5×5 são do tipo *same* para preservar a dimensão e permitir o empilhamento
- Pelo mesmo motivo, usa-se *padding* no *max-pool* (do tipo *same*) e *stride* $s = 1$
- Este é o **módulo inception**; Note que sua saída é $28 \times 28 \times 256$ no caso

Motivação

- Aplicamos, p.ex., uma conv 1x1, outra 3x3, outra 5x5, e ainda um *max-pooling*
- Então empilhamos todas as saídas em um bloco só
- As conv 3x3 e 5x5 são do tipo *same* para preservar a dimensão e permitir o empilhamento
- Pelo mesmo motivo, usa-se *padding* no *max-pool* (do tipo *same*) e *stride* $s = 1$
- Este é o **módulo inception**; Note que sua saída é 28x28x256 no caso

Motivação

- Aplicamos, p.ex., uma conv 1x1, outra 3x3, outra 5x5, e ainda um *max-pooling*
- Então empilhamos todas as saídas em um bloco só
- As conv 3x3 e 5x5 são do tipo *same* para preservar a dimensão e permitir o empilhamento
- Pelo mesmo motivo, usa-se *padding* no *max-pool* (do tipo *same*) e *stride s = 1*
- Este é o **módulo inception**; Note que sua saída é 28x28x256 no caso

Custo Computacional

- Ideia geral é deixar que a rede aprenda a melhor combinação possível de operações
- Porém, a inception ainda tem um custo computacional alto: vejamos p.ex. a conv 5x5
- Cada filtro tem dimensões 5x5x192; Na saída devemos calcular 28x28x32 números; Cada um desses números envolve 5x5x192 multiplicações

Custo Computacional

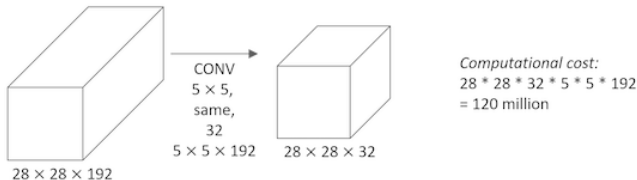
- Ideia geral é deixar que a rede aprenda a melhor combinação possível de operações
- Porém, a inception ainda tem um custo computacional alto: vejamos p.ex. a conv 5x5
- Cada filtro tem dimensões 5x5x192; Na saída devemos calcular 28x28x32 números; Cada um desses números envolve 5x5x192 multiplicações

Custo Computacional

- Ideia geral é deixar que a rede aprenda a melhor combinação possível de operações
- Porém, a inception ainda tem um custo computacional alto: vejamos p.ex. a conv 5×5
- Cada filtro tem dimensões $5 \times 5 \times 192$; Na saída devemos calcular $28 \times 28 \times 32$ números; Cada um desses números envolve $5 \times 5 \times 192$ multiplicações

Custo Computacional

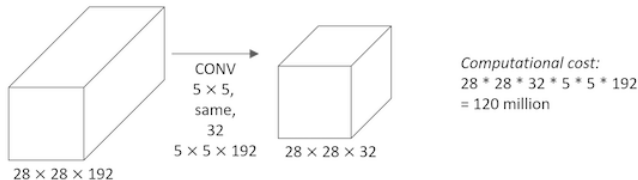
- Assim temos um total de 120M multiplicações!



- É aqui que entra a conv 1x1: reduzimos o número de canais na entrada para $28 \times 28 \times 16$ antes de aplicar conv 5×5

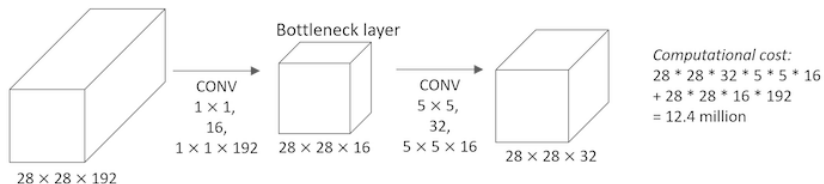
Custo Computacional

- Assim temos um total de 120M multiplicações!



- É aqui que entra a conv 1×1 : reduzimos o número de canais na entrada para $28 \times 28 \times 16$ antes de aplicar conv 5×5

Custo Computacional



- O bloco intermediário é chamado de “*bottleneck layer*” (analogia com o gargalo de uma garrafa)

Custo Computacional

- O número de multiplicações para gerar o bloco intermediário é $28 \times 28 \times 16 \times 192 = \sim 2.4\text{M}$
- Já para o bloco de saída temos $28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10.0\text{M}$
- A soma total é 12.4M; Cerca de um décimo dos 120M que tínhamos antes!
- NOTA 1: O número de adições é muito parecido com o número de multiplicações
- NOTA 2: Este *shrink* na *bottleneck layer* gera alguma perda de informação, mas na prática o desempenho da rede não é afetado

Custo Computacional

- O número de multiplicações para gerar o bloco intermediário é $28 \times 28 \times 16 \times 192 = \sim 2.4\text{M}$
- Já para o bloco de saída temos $28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10.0\text{M}$
- A soma total é 12.4M; Cerca de um décimo dos 120M que tínhamos antes!
- NOTA 1: O número de adições é muito parecido com o número de multiplicações
- NOTA 2: Este *shrink* na *bottleneck layer* gera alguma perda de informação, mas na prática o desempenho da rede não é afetado

Custo Computacional

- O número de multiplicações para gerar o bloco intermediário é $28 \times 28 \times 16 \times 192 = \sim 2.4\text{M}$
- Já para o bloco de saída temos $28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10.0\text{M}$
- A soma total é 12.4M; Cerca de um décimo dos 120M que tínhamos antes!
- NOTA 1: O número de adições é muito parecido com o número de multiplicações
- NOTA 2: Este *shrink* na *bottleneck layer* gera alguma perda de informação, mas na prática o desempenho da rede não é afetado

Custo Computacional

- O número de multiplicações para gerar o bloco intermediário é $28 \times 28 \times 16 \times 192 = \sim 2.4\text{M}$
- Já para o bloco de saída temos $28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10.0\text{M}$
- A soma total é 12.4M; Cerca de um décimo dos 120M que tínhamos antes!
- NOTA 1: O número de adições é muito parecido com o número de multiplicações
- NOTA 2: Este *shrink* na *bottleneck layer* gera alguma perda de informação, mas na prática o desempenho da rede não é afetado

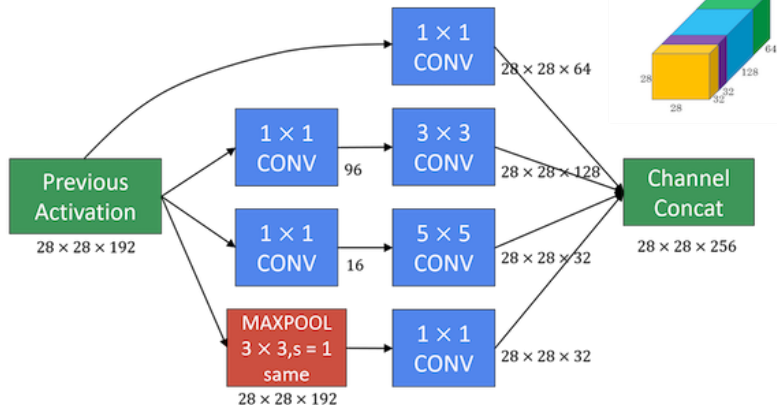
Custo Computacional

- O número de multiplicações para gerar o bloco intermediário é $28 \times 28 \times 16 \times 192 = \sim 2.4\text{M}$
- Já para o bloco de saída temos $28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10.0\text{M}$
- A soma total é 12.4M; Cerca de um décimo dos 120M que tínhamos antes!
- NOTA 1: O número de adições é muito parecido com o número de multiplicações
- NOTA 2: Este *shrink* na *bottleneck layer* gera alguma perda de informação, mas na prática o desempenho da rede não é afetado

Módulo Inception

Inception Module com entrada $28 \times 28 \times 192$:

Inception module



Módulo Inception

- Note que usamos o *bottleneck* não apenas na conv 5x5, mas na 3x3 também
- Já no *max-pool*, usando conv *same* e $s = 1$, temos uma saída com 192 canais
- Para reduzir isso, aplicamos 32 filtros 1x1, evitando ficar com tantos canais apenas do *max-pool*
- Por fim, as saídas de cada operação são todas concatenadas
- Isso é um módulo inception; Uma rede inception junta vários desses módulos!

Módulo Inception

- Note que usamos o *bottleneck* não apenas na conv 5x5, mas na 3x3 também
- Já no *max-pool*, usando conv *same* e $s = 1$, temos uma saída com 192 canais
- Para reduzir isso, aplicamos 32 filtros 1x1, evitando ficar com tantos canais apenas do *max-pool*
- Por fim, as saídas de cada operação são todas concatenadas
- Isso é um módulo inception; Uma rede inception junta vários desses módulos!

Módulo Inception

- Note que usamos o *bottleneck* não apenas na conv 5x5, mas na 3x3 também
- Já no *max-pool*, usando conv *same* e $s = 1$, temos uma saída com 192 canais
- Para reduzir isso, aplicamos 32 filtros 1x1, evitando ficar com tantos canais apenas do *max-pool*
- Por fim, as saídas de cada operação são todas concatenadas
- Isso é um módulo inception; Uma rede inception junta vários desses módulos!

Módulo Inception

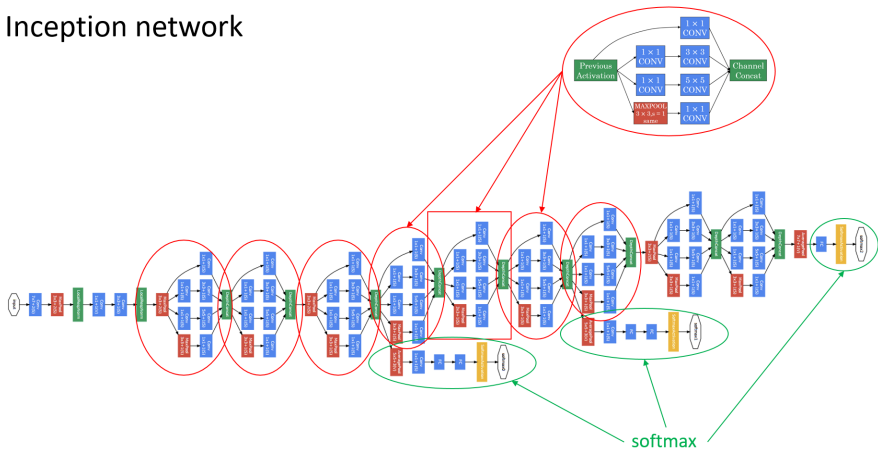
- Note que usamos o *bottleneck* não apenas na conv 5x5, mas na 3x3 também
- Já no *max-pool*, usando conv *same* e $s = 1$, temos uma saída com 192 canais
- Para reduzir isso, aplicamos 32 filtros 1x1, evitando ficar com tantos canais apenas do *max-pool*
- Por fim, as saídas de cada operação são todas concatenadas
- Isso é um módulo inception; Uma rede inception junta vários desses módulos!

Módulo Inception

- Note que usamos o *bottleneck* não apenas na conv 5x5, mas na 3x3 também
- Já no *max-pool*, usando conv *same* e $s = 1$, temos uma saída com 192 canais
- Para reduzir isso, aplicamos 32 filtros 1x1, evitando ficar com tantos canais apenas do *max-pool*
- Por fim, as saídas de cada operação são todas concatenadas
- Isso é um módulo inception; Uma rede inception junta vários desses módulos!

Rede Inception Completa

Inception network



Rede Inception Completa

- Em alguns pontos temos *pooling* (vermelho) antes de um módulo inception; Isso visa alterar dimensões
- A rede inception tem também alguns braços laterais, que tomam a saída de uma camada escondida e fazem uma predição usando algumas camadas FC e *softmax*
- Ideia é mostrar que mesmo saídas intermediárias da rede funcionam relativamente bem e isso tem também um efeito regularizador na rede, prevenindo *overfitting*

Rede Inception Completa

- Em alguns pontos temos *pooling* (vermelho) antes de um módulo inception; Isso visa alterar dimensões
- A rede inception tem também alguns braços laterais, que tomam a saída de uma camada escondida e fazem uma predição usando algumas camadas FC e *softmax*
- Ideia é mostrar que mesmo saídas intermediárias da rede funcionam relativamente bem e isso tem também um efeito regularizador na rede, prevenindo *overfitting*

Rede Inception Completa

- Em alguns pontos temos *pooling* (vermelho) antes de um módulo inception; Isso visa alterar dimensões
- A rede inception tem também alguns braços laterais, que tomam a saída de uma camada escondida e fazem uma predição usando algumas camadas FC e *softmax*
- Ideia é mostrar que mesmo saídas intermediárias da rede funcionam relativamente bem e isso tem também um efeito regularizador na rede, prevenindo *overfitting*

Curiosidades

- CURIOSIDADE 1: Os autores são Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke e Andrew Rabinovich
- CURIOSIDADE 2: Eles são do Google e chamaram essa rede de GoogLeNet (uma referência à LeNet)

Curiosidades

- CURIOSIDADE 1: Os autores são Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke e Andrew Rabinovich
- CURIOSIDADE 2: Eles são do Google e chamaram essa rede de GoogLeNet (uma referência à LeNet)

Curiosidades

- CURIOSIDADE 3: O nome “inception” vem de um meme, do filme “The Inception” e a frase “We need to go deeper”!



<http://knowyourmeme.com/memes/we-need-to-go-deeper>



- NOTA: Temos versões mais modernas do modelo, Inception v2, v3, etc., incluindo combinações com ideias da ResNet; Mas todas elas se baseiam no módulo inception

Curiosidades

- CURIOSIDADE 3: O nome “inception” vem de um meme, do filme “The Inception” e a frase “We need to go deeper”!



<http://knowyourmeme.com/memes/we-need-to-go-deeper>



- NOTA: Temos versões mais modernas do modelo, Inception v2, v3, etc., incluindo combinações com ideias da ResNet; Mas todas elas se baseiam no módulo inception

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet**
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

MobileNet⁷

- Modelos vistos até agora são exigentes computacionalmente
- A MobileNet pode ser rodada até mesmo em um smartphone ou sistema embarcado
- Baixo custo de *deployment*
- Ideia principal: **depthwise-separable convolutions**

⁷Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.

MobileNet⁷

- Modelos vistos até agora são exigentes computacionalmente
- A MobileNet pode ser rodada até mesmo em um smartphone ou sistema embarcado
- Baixo custo de *deployment*
- Ideia principal: **depthwise-separable convolutions**

⁷Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.

MobileNet⁷

- Modelos vistos até agora são exigentes computacionalmente
- A MobileNet pode ser rodada até mesmo em um smartphone ou sistema embarcado
- Baixo custo de *deployment*
- Ideia principal: **depthwise-separable convolutions**

⁷Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.

MobileNet⁷

- Modelos vistos até agora são exigentes computacionalmente
- A MobileNet pode ser rodada até mesmo em um smartphone ou sistema embarcado
- Baixo custo de *deployment*
- Ideia principal: **depthwise-separable convolutions**

⁷Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.

Depthwise-separable Convolutions

- Veja um exemplo abaixo da convolução normal (sem *padding*):

Diagram illustrating a normal 3D convolution operation:

Input: $6 \times 6 \times 3$ ($n \times n \times n_c$)

Filter: $3 \times 3 \times 3$ ($f \times f \times n_c$)

Output: $4 \times 4 \times 5$ ($n_{out} \times n_{out} \times n_c$)

Computational cost calculation:

$$\text{Computational cost} = \frac{\# \text{filter params}}{3 \times 3 \times 3} \times \frac{\# \text{filter positions}}{4 \times 4} \times \# \text{of filters}$$

Handwritten calculation: $\rightarrow 2160$

- Temos aqui 2160 multiplicações
- A *depthwise-separable convolution* gera uma saída com essas mesmas dimensões, porém com menos cálculos

Depthwise-separable Convolutions

- Veja um exemplo abaixo da convolução normal (sem *padding*):

Computational cost = $\frac{\text{\#filter params}}{3 \times 3 \times 3} \times \frac{\text{\#filter positions}}{4 \times 4} \times \text{\# of filters}$

$\rightarrow \underline{2160} = \frac{3 \times 3 \times 3}{\uparrow} \times \frac{4 \times 4}{\uparrow} \times \frac{5}{\uparrow}$

- Temos aqui 2160 multiplicações
- A *depthwise-separable convolution* gera uma saída com essas mesmas dimensões, porém com menos cálculos

Depthwise-separable Convolutions

- Veja um exemplo abaixo da convolução normal (sem *padding*):

Computational cost = $\frac{\text{\#filter params}}{3 \times 3 \times 3} \times \frac{\text{\#filter positions}}{4 \times 4} \times \text{\# of filters}$

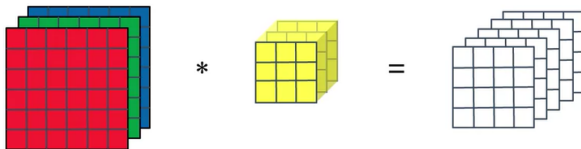
$\rightarrow \underline{2160} = \frac{3 \times 3 \times 3}{3 \times 3 \times 3} \times \frac{4 \times 4}{4 \times 4} \times 5$

- Temos aqui 2160 multiplicações
- A *depthwise-separable convolution* gera uma saída com essas mesmas dimensões, porém com menos cálculos

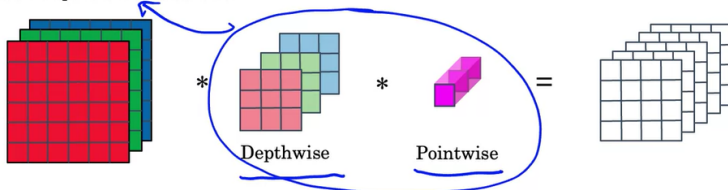
Depthwise-separable Convolution

- Dois passos: *depthwise* e *pointwise convolution*

Normal Convolution



Depthwise Separable Convolution



Depthwise Convolution

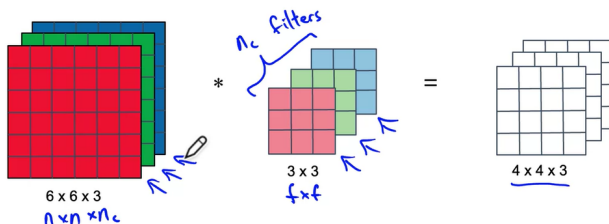
- Na *depthwise convolution*, o filtro é $f \times f$ (não $f \times f \times n_c$ como antes)
- O número de filtros é n_c (3 no caso)
- E se aplica cada um desses filtros a cada um dos canais de entrada

Depthwise Convolution

- Na *depthwise convolution*, o filtro é $f \times f$ (não $f \times f \times n_c$ como antes)
- O número de filtros é n_c (3 no caso)
- E se aplica cada um desses filtros a cada um dos canais de entrada

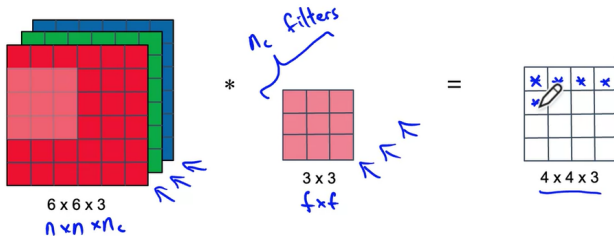
Depthwise Convolution

- Na *depthwise convolution*, o filtro é $f \times f$ (não $f \times f \times n_c$ como antes)
- O número de filtros é n_c (3 no caso)
- E se aplica cada um desses filtros a cada um dos canais de entrada



Depthwise Convolution

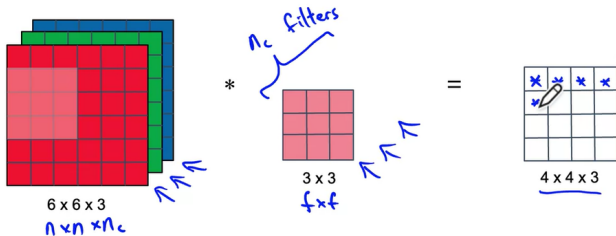
No canal vermelho:



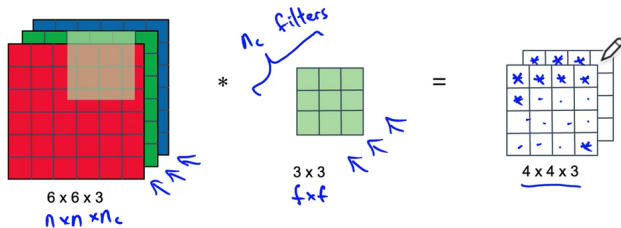
No verde:

Depthwise Convolution

No canal vermelho:

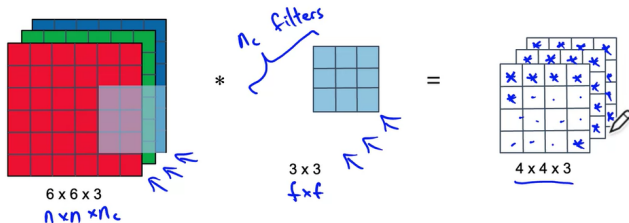


No verde:



Depthwise Convolution

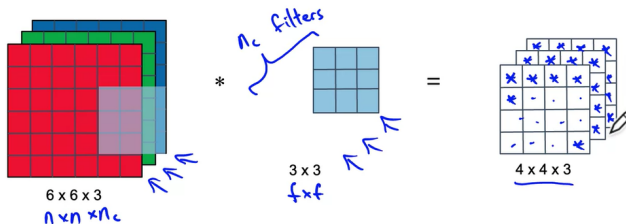
No azul:



- O tamanho da saída é $n_{out} \times n_{out} \times n_c$, como na convolução normal com 1 filtro
- Porém com muito menos multiplicações (ver a seguir)!

Depthwise Convolution

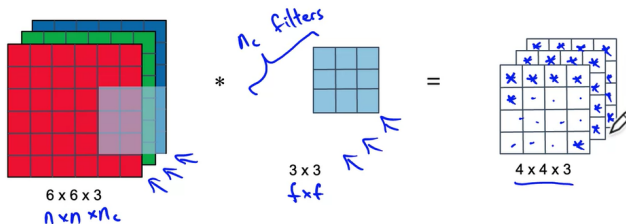
No azul:



- O tamanho da saída é $n_{out} \times n_{out} \times n_c$, como na convolução normal com 1 filtro
- Porém com muito menos multiplicações (ver a seguir)!

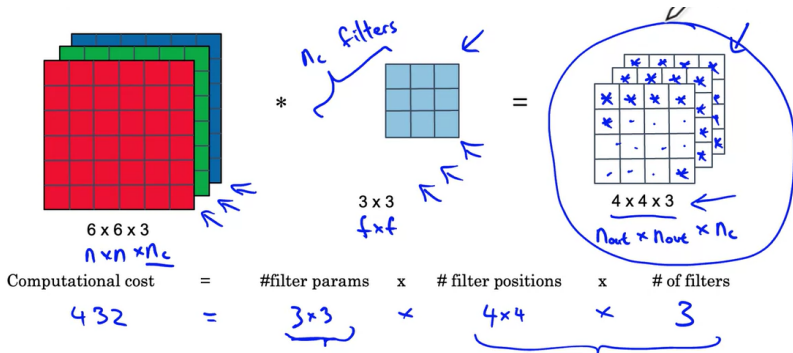
Depthwise Convolution

No azul:



- O tamanho da saída é $n_{out} \times n_{out} \times n_c$, como na convolução normal com 1 filtro
- Porém com muito menos multiplicações (ver a seguir)!

Depthwise Convolution - Número de Multiplicações



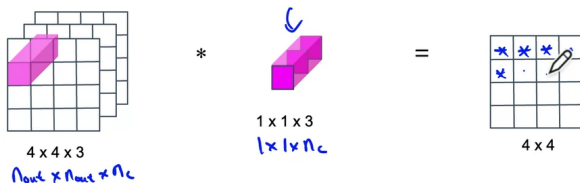
O número de multiplicações no exemplo acima passa a ser 432

Pointwise Convolution

- Mas precisamos ainda de um passo a mais para obter a saída $4 \times 4 \times 5$ que desejamos: **pointwise convolution**
- Vamos convoluir a saída da *depthwise* com um filtro $1 \times 1 \times n_c$
- Mas isso gera apenas uma saída 4×4 ; Aplicamos então n'_c filtros (no nosso caso 5)

Pointwise Convolution

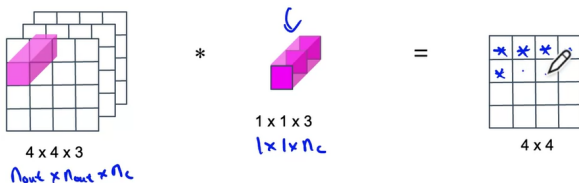
- Mas precisamos ainda de um passo a mais para obter a saída $4 \times 4 \times 5$ que desejamos: **pointwise convolution**
- Vamos convoluir a saída da *depthwise* com um filtro $1 \times 1 \times n_c$



- Mas isso gera apenas uma saída 4×4 ; Aplicamos então n'_c filtros (no nosso caso 5)

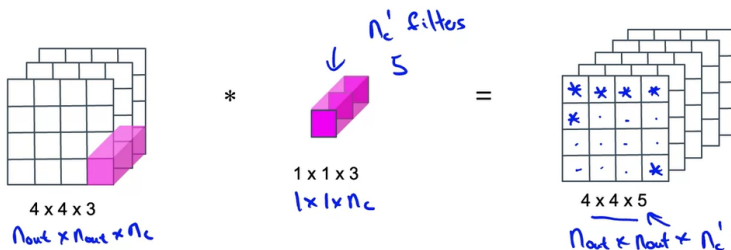
Pointwise Convolution

- Mas precisamos ainda de um passo a mais para obter a saída $4 \times 4 \times 5$ que desejamos: **pointwise convolution**
- Vamos convoluir a saída da *depthwise* com um filtro $1 \times 1 \times n_c$



- Mas isso gera apenas uma saída 4×4 ; Aplicamos então n'_c filtros (no nosso caso 5)

Pointwise Convolution-Multiplicações



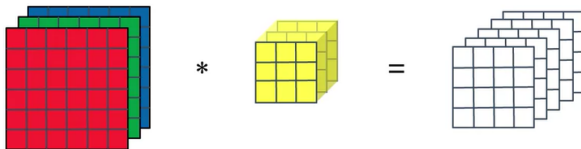
$$\begin{array}{rclclclcl}
 \text{Computational cost} & = & \text{\#filter params} & \times & \text{\# filter positions} & \times & \text{\# of filters} \\
 240 & = & 1 \times 1 \times 3 & \times & 4 \times 4 & \times & 5
 \end{array}$$

O número de multiplicações dessa operação é 240

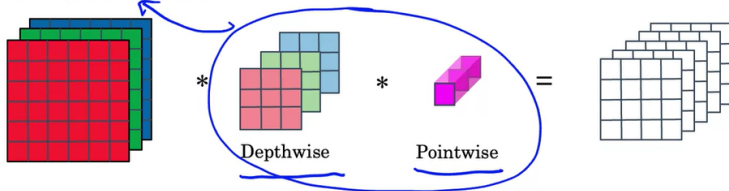
Depthwise-separable Convolution

- Note que as dimensões de entrada e saída são as mesmas nos dois tipos de convolução:

Normal Convolution



Depthwise Separable Convolution



- Mas o custo em termos de multiplicações é bem menor (cerca de 31% da convolução clássica)

Depthwise-separable Convolution - Custo Computacional

Cost of normal convolution

2160

Cost of depthwise separable convolution

$$\begin{array}{ccc} \text{depthwise} & + & \text{pointwise} \\ 432 & + & 240 = 672 \end{array}$$

$$\frac{672}{2160} = 0.31 \leftarrow$$

$$= \frac{1}{\underbrace{n_c}_{\frac{1}{5}}} + \frac{1}{f^2}$$

$$= \frac{1}{512} + \frac{1}{3^2}$$

~ 10 times cheaper

- Em geral, a razão entre o número de multiplicações na convolução *depthwise-separable* e na normal é $(1/n'_c + 1/f^2)$
- n'_c costuma ser bem grande (e.g., 512) e no caso a razão seria $\sim 10x$ mais barata computacionalmente na inferência

Depthwise-separable Convolution - Custo Computacional

Cost of normal convolution $\swarrow$ 2160

Cost of depthwise separable convolution $\swarrow$
 depthwise + pointwise
 432 + 240 = 672

$$\frac{672}{2160} = 0.31 \leftarrow$$

$$\begin{aligned}
 &= \frac{1}{n_c} + \frac{1}{f^2} \\
 &\quad \frac{1}{5} + \frac{1}{9} \\
 &= \frac{1}{512} + \frac{1}{3^2} \\
 &\quad \quad \quad \nwarrow \quad \quad \quad \nwarrow \\
 &\quad \quad \sim 10 \text{ times cheaper}
 \end{aligned}$$

- Em geral, a razão entre o número de multiplicações na convolução *depthwise-separable* e na normal é $(1/n'_c + 1/f^2)$
- n'_c costuma ser bem grande (e.g., 512) e no caso a razão seria $\sim 10\times$ mais barata computacionalmente na inferência

Depthwise-separable Convolution - Custo Computacional

Cost of normal convolution $\swarrow$ 2160

Cost of depthwise separable convolution $\swarrow$
 depthwise + pointwise
 432 + 240 = 672

$$\frac{672}{2160} = 0.31 \leftarrow$$

$$\begin{aligned}
 &= \frac{1}{n_c'} + \frac{1}{f^2} \\
 &\quad \frac{1}{5} + \frac{1}{9} \\
 &= \frac{1}{512} + \frac{1}{3^2} \\
 &\quad \quad \quad \nwarrow \quad \quad \quad \nwarrow \\
 &\quad \quad \quad \sim 10 \text{ times cheaper}
 \end{aligned}$$

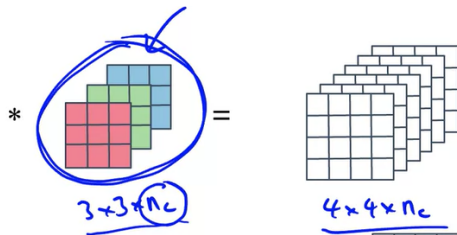
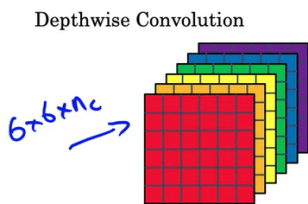
- Em geral, a razão entre o número de multiplicações na convolução *depthwise-separable* e na normal é $(1/n_c' + 1/f^2)$
- n_c' costuma ser bem grande (e.g., 512) e no caso a razão seria $\sim 10x$ mais barata computacionalmente na inferência

Depthwise-separable Convolution

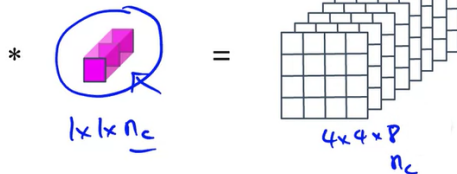
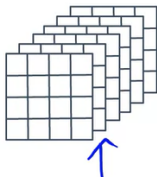
- NOTA: A *depthwise convolution* funciona para qualquer número de canais
- Porém, nos nossos diagramas, o filtro $3 \times 3 \times n_c$ é simbolizado sempre por 3 matrizes
- Isso é apenas um ícone para indicar *depthwise convolution*
- Mesma ideia para o filtro $1 \times 1 \times n_c$ (rosa): denotado como se tivesse 3 valores apenas

Depthwise-separable Convolution

Depthwise Convolution



Pointwise Convolution



Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet**
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

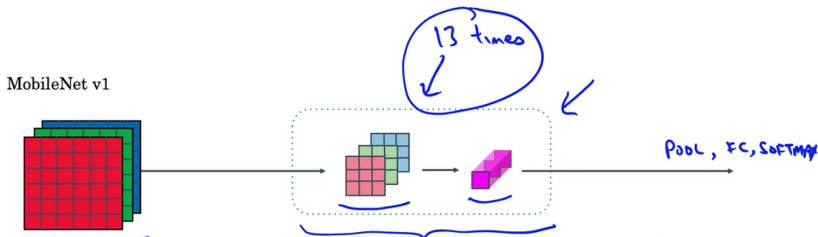
MobileNet⁸

- Ideia é substituir uma convolução cara computacionalmente por uma bem mais barata
- Artigo da versão v1 usa 13 blocos *depthwise-separable*, seguidos por camadas de *pool*, FC e softmax
- Funciona bem, usando muito menos computação do que modelos baseados na convolução clássica

⁸Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks

MobileNet⁸

- Ideia é substituir uma convolução cara computacionalmente por uma bem mais barata
- Artigo da versão v1 usa 13 blocos *depthwise-separable*, seguidos por camadas de *pool*, FC e softmax

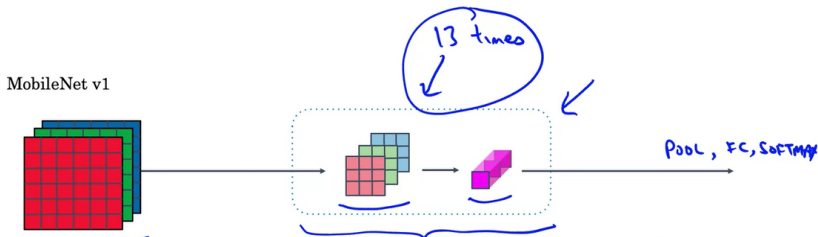


- Funciona bem, usando muito menos computação do que modelos baseados na convolução clássica

⁸Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks

MobileNet⁸

- Ideia é substituir uma convolução cara computacionalmente por uma bem mais barata
- Artigo da versão v1 usa 13 blocos *depthwise-separable*, seguidos por camadas de *pool*, FC e softmax



- Funciona bem, usando muito menos computação do que modelos baseados na convolução clássica

⁸Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks

MobileNetV2

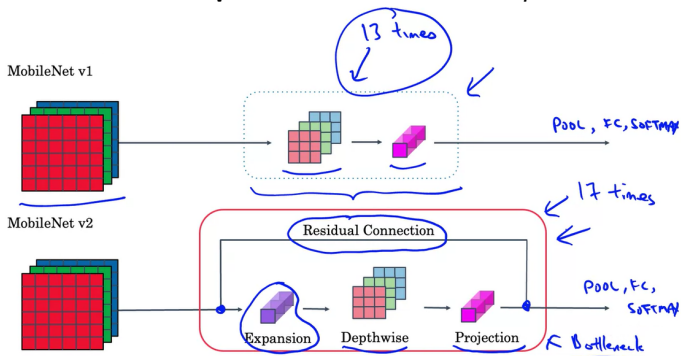
- Na versão v2, temos duas melhorias
- A 1ª é uma conexão residual idêntica à da ResNet, somando a saída antes dos blocos *depthwise* com a saída após
- A 2ª é uma **camada de expansão** antes da conv *depthwise*

MobileNetV2

- Na versão v2, temos duas melhorias
- A 1ª é uma conexão residual idêntica à da ResNet, somando a saída antes dos blocos *depthwise* com a saída após
- A 2ª é uma **camada de expansão** antes da conv *depthwise*

MobileNetV2

- Na versão v2, temos duas melhorias
- A 1ª é uma conexão residual idêntica à da ResNet, somando a saída antes dos blocos *depthwise* com a saída após
- A 2ª é uma **camada de expansão** antes da conv *depthwise*



MobileNetV2

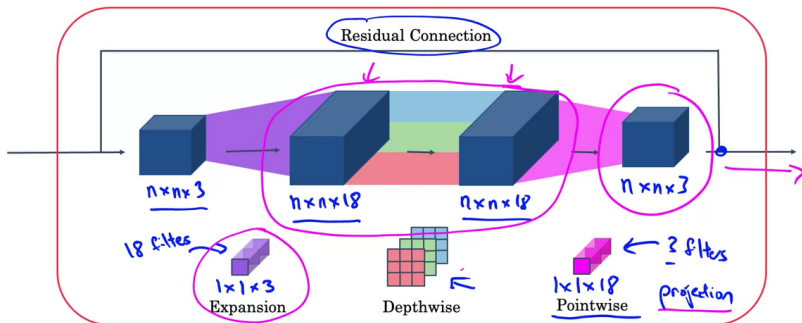
- Note que agora a operação *pointwise* é chamada de “*projection*”
- O bloco todo, incluindo expansão e residual, é repetido 17 vezes e chamado de *bottleneck*
- Vejamos em detalhe esse bloco:

MobileNetV2

- Note que agora a operação *pointwise* é chamada de “*projection*”
- O bloco todo, incluindo expansão e residual, é repetido 17 vezes e chamado de *bottleneck*
- Vejamos em detalhe esse bloco:

MobileNetV2

- Note que agora a operação *pointwise* é chamada de “*projection*”
- O bloco todo, incluindo expansão e residual, é repetido 17 vezes e chamado de *bottleneck*
- Vejamos em detalhe esse bloco:



Bloco Bottleneck

- A expansão consiste em aplicar conv $1 \times 1 \times n_c$, porém com um grande número de filtros (usualmente 18)
- No caso, o tensor é expandido por um fator de 6 (de 3 para 18 canais)
- Em seguida temos a *depthwise convolution*, que mantém dimensões $n \times n \times 18$ usando *padding*
- Por fim, aplica-se a *pointwise convolution*; Usa-se, p.ex., $n_c' = 3$ filtros
- Essa operação na MobileNet v2 é chamada de *projection* porque projeta um tensor $n \times n \times 18$ para um $n \times n \times 3$

Bloco Bottleneck

- A expansão consiste em aplicar conv $1 \times 1 \times n_c$, porém com um grande número de filtros (usualmente 18)
- No caso, o tensor é expandido por um fator de 6 (de 3 para 18 canais)
- Em seguida temos a *depthwise convolution*, que mantém dimensões $n \times n \times 18$ usando *padding*
- Por fim, aplica-se a *pointwise convolution*; Usa-se, p.ex., $n_c' = 3$ filtros
- Essa operação na MobileNet v2 é chamada de *projection* porque projeta um tensor $n \times n \times 18$ para um $n \times n \times 3$

Bloco Bottleneck

- A expansão consiste em aplicar conv $1 \times 1 \times n_c$, porém com um grande número de filtros (usualmente 18)
- No caso, o tensor é expandido por um fator de 6 (de 3 para 18 canais)
- Em seguida temos a *depthwise convolution*, que mantém dimensões $n \times n \times 18$ usando *padding*
- Por fim, aplica-se a *pointwise convolution*; Usa-se, p.ex., $n_c' = 3$ filtros
- Essa operação na MobileNet v2 é chamada de *projection* porque projeta um tensor $n \times n \times 18$ para um $n \times n \times 3$

Bloco Bottleneck

- A expansão consiste em aplicar conv $1 \times 1 \times n_c$, porém com um grande número de filtros (usualmente 18)
- No caso, o tensor é expandido por um fator de 6 (de 3 para 18 canais)
- Em seguida temos a *depthwise convolution*, que mantém dimensões $n \times n \times 18$ usando *padding*
- Por fim, aplica-se a *pointwise convolution*; Usa-se, p.ex., $n_c' = 3$ filtros
- Essa operação na MobileNet v2 é chamada de *projection* porque projeta um tensor $n \times n \times 18$ para um $n \times n \times 3$

Bloco Bottleneck

- A expansão consiste em aplicar conv $1 \times 1 \times n_c$, porém com um grande número de filtros (usualmente 18)
- No caso, o tensor é expandido por um fator de 6 (de 3 para 18 canais)
- Em seguida temos a *depthwise convolution*, que mantém dimensões $n \times n \times 18$ usando *padding*
- Por fim, aplica-se a *pointwise convolution*; Usa-se, p.ex., $n_c' = 3$ filtros
- Essa operação na MobileNet v2 é chamada de *projection* porque projeta um tensor $n \times n \times 18$ para um $n \times n \times 3$

Bloco Bottleneck

- Mas afinal, por que precisamos desse bloco?
- Note que a *expansion* aumenta o tamanho da representação, permitindo à rede aprender uma função mais rica
- Porém, em um smartphone, temos restrições de memória
- A operação de projeção reduz a quantidade de informação passada ao próximo bloco, e que precisa ser armazenada
- Em suma, a rede aprende padrões complexos, mesmo com uso reduzido de memória

Bloco Bottleneck

- Mas afinal, por que precisamos desse bloco?
- Note que a *expansion* aumenta o tamanho da representação, permitindo à rede aprender uma função mais rica
- Porém, em um smartphone, temos restrições de memória
- A operação de projeção reduz a quantidade de informação passada ao próximo bloco, e que precisa ser armazenada
- Em suma, a rede aprende padrões complexos, mesmo com uso reduzido de memória

Bloco Bottleneck

- Mas afinal, por que precisamos desse bloco?
- Note que a *expansion* aumenta o tamanho da representação, permitindo à rede aprender uma função mais rica
- Porém, em um smartphone, temos restrições de memória
- A operação de projeção reduz a quantidade de informação passada ao próximo bloco, e que precisa ser armazenada
- Em suma, a rede aprende padrões complexos, mesmo com uso reduzido de memória

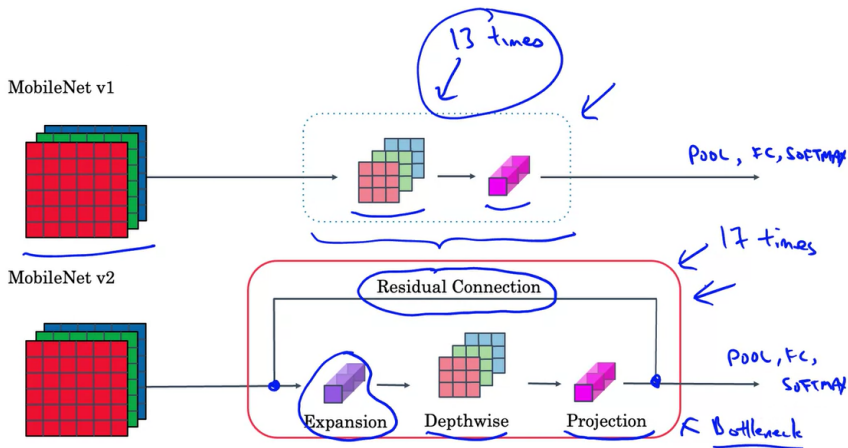
Bloco Bottleneck

- Mas afinal, por que precisamos desse bloco?
- Note que a *expansion* aumenta o tamanho da representação, permitindo à rede aprender uma função mais rica
- Porém, em um smartphone, temos restrições de memória
- A operação de projeção reduz a quantidade de informação passada ao próximo bloco, e que precisa ser armazenada
- Em suma, a rede aprende padrões complexos, mesmo com uso reduzido de memória

Bloco Bottleneck

- Mas afinal, por que precisamos desse bloco?
- Note que a *expansion* aumenta o tamanho da representação, permitindo à rede aprender uma função mais rica
- Porém, em um smartphone, temos restrições de memória
- A operação de projeção reduz a quantidade de informação passada ao próximo bloco, e que precisa ser armazenada
- Em suma, a rede aprende padrões complexos, mesmo com uso reduzido de memória

Arquitetura Geral



Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet**
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

EfficientNet⁹

- Como ajustar a MobileNet de modo que, em um aparelho mais potente, eu pudesse rodar um modelo maior que funcionaria um pouco melhor?
- Essa é a motivação da EfficientNet!
- Seja uma rede que recebe uma imagem com resolução r , tem profundidade d e largura de camada w
- Os autores mostram três possibilidades para escalar o modelo para mais ou para menos
- A primeira é aumentar a resolução (azul difuso em volta na figura a seguir)

⁹Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.

EfficientNet⁹

- Como ajustar a MobileNet de modo que, em um aparelho mais potente, eu pudesse rodar um modelo maior que funcionaria um pouco melhor?
- Essa é a motivação da EfficientNet!
- Seja uma rede que recebe uma imagem com resolução r , tem profundidade d e largura de camada w
- Os autores mostram três possibilidades para escalar o modelo para mais ou para menos
- A primeira é aumentar a resolução (azul difuso em volta na figura a seguir)

⁹Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.

EfficientNet⁹

- Como ajustar a MobileNet de modo que, em um aparelho mais potente, eu pudesse rodar um modelo maior que funcionaria um pouco melhor?
- Essa é a motivação da EfficientNet!
- Seja uma rede que recebe uma imagem com resolução r , tem profundidade d e largura de camada w
- Os autores mostram três possibilidades para escalar o modelo para mais ou para menos
- A primeira é aumentar a resolução (azul difuso em volta na figura a seguir)

⁹Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.

EfficientNet⁹

- Como ajustar a MobileNet de modo que, em um aparelho mais potente, eu pudesse rodar um modelo maior que funcionaria um pouco melhor?
- Essa é a motivação da EfficientNet!
- Seja uma rede que recebe uma imagem com resolução r , tem profundidade d e largura de camada w
- Os autores mostram três possibilidades para escalar o modelo para mais ou para menos
- A primeira é aumentar a resolução (azul difuso em volta na figura a seguir)

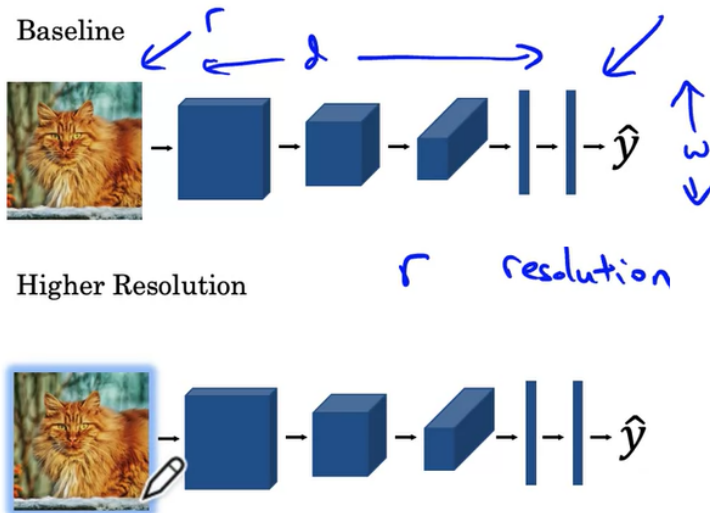
⁹Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.

EfficientNet⁹

- Como ajustar a MobileNet de modo que, em um aparelho mais potente, eu pudesse rodar um modelo maior que funcionaria um pouco melhor?
- Essa é a motivação da EfficientNet!
- Seja uma rede que recebe uma imagem com resolução r , tem profundidade d e largura de camada w
- Os autores mostram três possibilidades para escalar o modelo para mais ou para menos
- A primeira é aumentar a resolução (azul difuso em volta na figura a seguir)

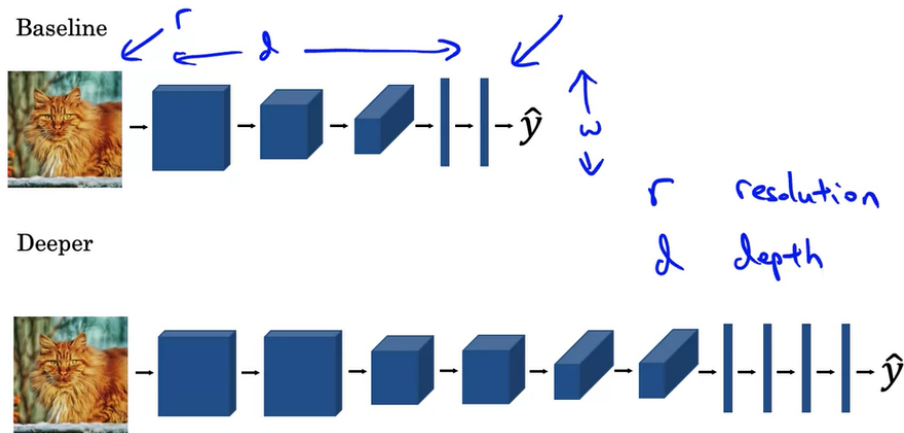
⁹Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.

EfficientNet



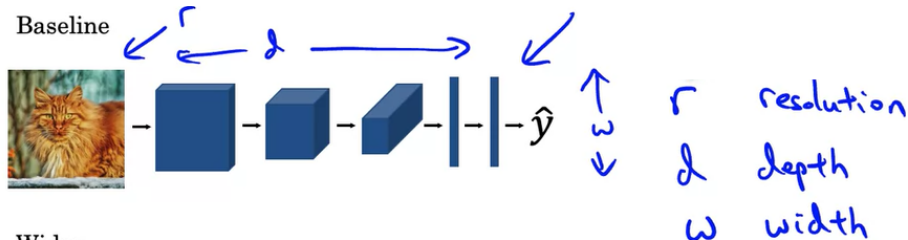
EfficientNet

Outra seria mexer na profundidade:

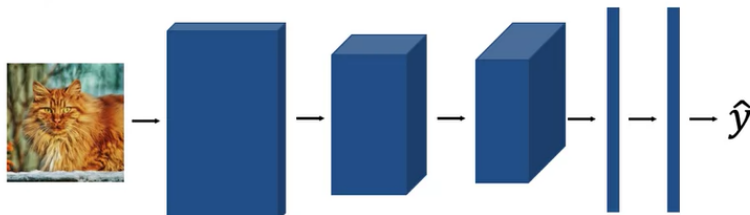


EfficientNet

Ou tornar as camadas mais largas:

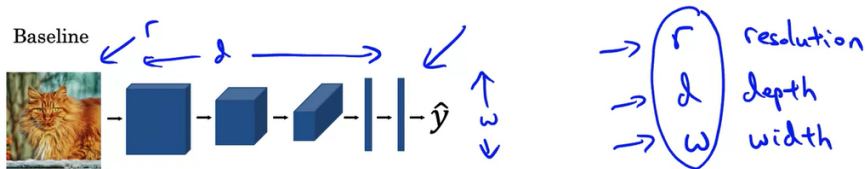


Wider

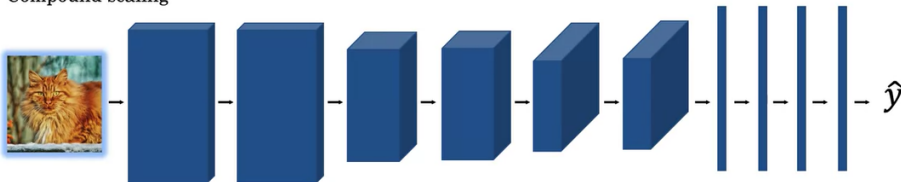


EfficientNet

Ou então pode-se usar o *compound scaling*, em que se aumenta ou diminui r , d e w simultaneamente:



Compound scaling



EfficientNet

- Pergunta: dado um conjunto específico de hardware, qual a melhor escolha para r , d e w ?
- A EfficientNet oferece então diferentes combinações dessas configurações para tipos diferentes de hardware

EfficientNet

- Pergunta: dado um conjunto específico de hardware, qual a melhor escolha para r , d e w ?
- A EfficientNet oferece então diferentes combinações dessas configurações para tipos diferentes de hardware

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning**
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

Implementações Open-Source

- Replicar as arquiteturas que vimos apenas lendo o artigo é bem difícil (muitos detalhes importantes)
- Todos esses modelos têm códigos disponíveis publicamente
- Familiarize-se com o GitHub!
- Se possível, veja o código dos autores originais
- À medida que for evoluindo, contribua com seus códigos também!

Implementações Open-Source

- Replicar as arquiteturas que vimos apenas lendo o artigo é bem difícil (muitos detalhes importantes)
- Todos esses modelos têm códigos disponíveis publicamente
- Familiarize-se com o GitHub!
- Se possível, veja o código dos autores originais
- À medida que for evoluindo, contribua com seus códigos também!

Implementações Open-Source

- Replicar as arquiteturas que vimos apenas lendo o artigo é bem difícil (muitos detalhes importantes)
- Todos esses modelos têm códigos disponíveis publicamente
- Familiarize-se com o GitHub!
- Se possível, veja o código dos autores originais
- À medida que for evoluindo, contribua com seus códigos também!

Implementações Open-Source

- Replicar as arquiteturas que vimos apenas lendo o artigo é bem difícil (muitos detalhes importantes)
- Todos esses modelos têm códigos disponíveis publicamente
- Familiarize-se com o GitHub!
- Se possível, veja o código dos autores originais
- À medida que for evoluindo, contribua com seus códigos também!

Implementações Open-Source

- Replicar as arquiteturas que vimos apenas lendo o artigo é bem difícil (muitos detalhes importantes)
- Todos esses modelos têm códigos disponíveis publicamente
- Familiarize-se com o GitHub!
- Se possível, veja o código dos autores originais
- À medida que for evoluindo, contribua com seus códigos também!

Transfer Learning

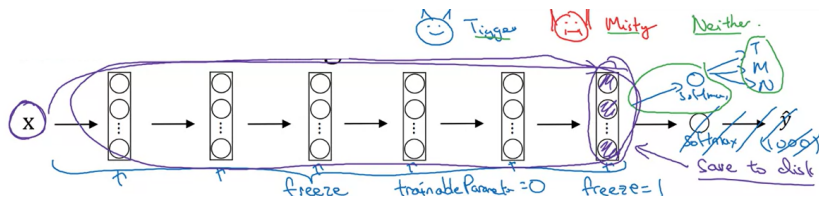
- Na sua própria aplicação de visão computacional, em vez de treinar do zero inicializando pesos aleatórios, é melhor partir de pesos que outras pessoas já treinaram
- Isso já foi feito em bases enormes como ImageNet, MS COCO, Pascal, etc., gastando muito tempo em várias GPUs poderosas
- Imagine um detector de gatos com 3 classes: “Tigger”, “Misty” e “Neither”:

Transfer Learning

- Na sua própria aplicação de visão computacional, em vez de treinar do zero inicializando pesos aleatórios, é melhor partir de pesos que outras pessoas já treinaram
- Isso já foi feito em bases enormes como ImageNet, MS COCO, Pascal, etc., gastando muito tempo em várias GPUs poderosas
- Imagine um detector de gatos com 3 classes: “Tigger”, “Misty” e “Neither”:

Transfer Learning

- Na sua própria aplicação de visão computacional, em vez de treinar do zero inicializando pesos aleatórios, é melhor partir de pesos que outras pessoas já treinaram
- Isso já foi feito em bases enormes como ImageNet, MS COCO, Pascal, etc., gastando muito tempo em várias GPUs poderosas
- Imagine um detector de gatos com 3 classes: “Tigger”, “Misty” e “Neither”:



Transfer Learning

- Provavelmente seu conjunto de treino, com gatos “Tigger” e “Misty”, vai ser pequeno
- Recomenda-se então procurar por uma implementação *open-source* de uma rede e baixar não apenas os códigos, mas também os pesos treinados, p.ex., na ImageNet
- IMPORTANTE: A ImageNet tem 1000 classes, no caso então teria que trocar a camada *softmax* de 1000 classes para a sua própria
- E apenas os parâmetros da camada *softmax* seriam aprendidos; Todos os demais seriam **congelados**
- Isso deve resultar em bons resultados mesmo com um treino muito menor

Transfer Learning

- Provavelmente seu conjunto de treino, com gatos “Tigger” e “Misty”, vai ser pequeno
- Recomenda-se então procurar por uma implementação *open-source* de uma rede e baixar não apenas os códigos, mas também os pesos treinados, p.ex., na ImageNet
- IMPORTANTE: A ImageNet tem 1000 classes, no caso então teria que trocar a camada *softmax* de 1000 classes para a sua própria
- E apenas os parâmetros da camada *softmax* seriam aprendidos; Todos os demais seriam **congelados**
- Isso deve resultar em bons resultados mesmo com um treino muito menor

Transfer Learning

- Provavelmente seu conjunto de treino, com gatos “Tigger” e “Misty”, vai ser pequeno
- Recomenda-se então procurar por uma implementação *open-source* de uma rede e baixar não apenas os códigos, mas também os pesos treinados, p.ex., na ImageNet
- IMPORTANTE: A ImageNet tem 1000 classes, no caso então teria que trocar a camada *softmax* de 1000 classes para a sua própria
- E apenas os parâmetros da camada *softmax* seriam aprendidos; Todos os demais seriam **congelados**
- Isso deve resultar em bons resultados mesmo com um treino muito menor

Transfer Learning

- Provavelmente seu conjunto de treino, com gatos “Tigger” e “Misty”, vai ser pequeno
- Recomenda-se então procurar por uma implementação *open-source* de uma rede e baixar não apenas os códigos, mas também os pesos treinados, p.ex., na ImageNet
- IMPORTANTE: A ImageNet tem 1000 classes, no caso então teria que trocar a camada *softmax* de 1000 classes para a sua própria
- E apenas os parâmetros da camada *softmax* seriam aprendidos; Todos os demais seriam **congelados**
- Isso deve resultar em bons resultados mesmo com um treino muito menor

Transfer Learning

- Provavelmente seu conjunto de treino, com gatos “Tigger” e “Misty”, vai ser pequeno
- Recomenda-se então procurar por uma implementação *open-source* de uma rede e baixar não apenas os códigos, mas também os pesos treinados, p.ex., na ImageNet
- IMPORTANTE: A ImageNet tem 1000 classes, no caso então teria que trocar a camada *softmax* de 1000 classes para a sua própria
- E apenas os parâmetros da camada *softmax* seriam aprendidos; Todos os demais seriam **congelados**
- Isso deve resultar em bons resultados mesmo com um treino muito menor

Transfer Learning

- NOTA: Os *frameworks* de *deep learning* suportam isso através de parâmetros como *trainableParameter=0* ou *freeze=1*
- Um bom truque aqui é pré-calcular as ativações na parte congelada para cada x no treinamento e salvar em disco, já que essa parte age como uma função fixa
- Assim não vai precisar recalcular isso em cada época do aprendizado da softmax.

Transfer Learning

- NOTA: Os *frameworks* de *deep learning* suportam isso através de parâmetros como *trainableParameter=0* ou *freeze=1*
- Um bom truque aqui é pré-calcular as ativações na parte congelada para cada x no treinamento e salvar em disco, já que essa parte age como uma função fixa
- Assim não vai precisar recalcular isso em cada época do aprendizado da softmax.

Transfer Learning

- NOTA: Os *frameworks* de *deep learning* suportam isso através de parâmetros como *trainableParameter=0* ou *freeze=1*
- Um bom truque aqui é pré-calcular as ativações na parte congelada para cada x no treinamento e salvar em disco, já que essa parte age como uma função fixa
- Assim não vai precisar recalcular isso em cada época do aprendizado da softmax.

Transfer Learning - Congelamento Parcial

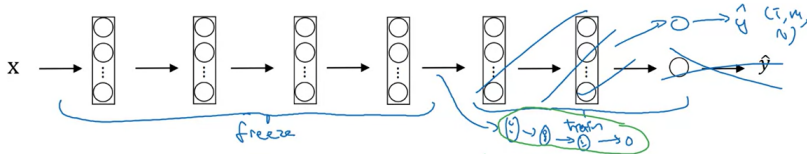
- E se o conjunto de treinamento for grande?
- Neste caso, pode-se congelar apenas as primeiras camadas e treinar as seguintes
- Isso sempre ajustando a camada de saída para o seu problema
- Nas camadas treináveis, pode inicializar o gradiente descendente com os pesos pré-treinados ou descartar totalmente aquelas camadas e usar outras
- Em geral, quanto mais dado para treinar, menos camadas congeladas!

Transfer Learning - Congelamento Parcial

- E se o conjunto de treinamento for grande?
- Neste caso, pode-se congelar apenas as primeiras camadas e treinar as seguintes
- Isso sempre ajustando a camada de saída para o seu problema
- Nas camadas treináveis, pode inicializar o gradiente descendente com os pesos pré-treinados ou descartar totalmente aquelas camadas e usar outras
- Em geral, quanto mais dado para treinar, menos camadas congeladas!

Transfer Learning - Congelamento Parcial

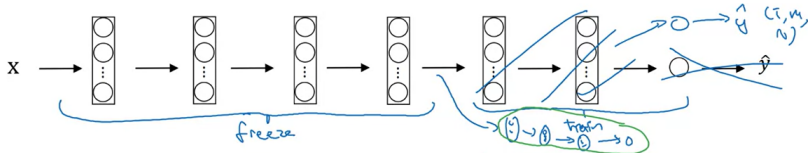
- E se o conjunto de treinamento for grande?
- Neste caso, pode-se congelar apenas as primeiras camadas e treinar as seguintes
- Isso sempre ajustando a camada de saída para o seu problema



- Nas camadas treináveis, pode inicializar o gradiente descendente com os pesos pré-treinados ou descartar totalmente aquelas camadas e usar outras
- Em geral, quanto mais dado para treinar, menos camadas congeladas!

Transfer Learning - Congelamento Parcial

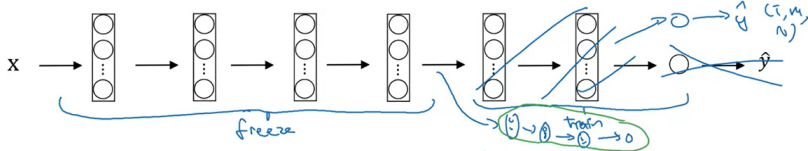
- E se o conjunto de treinamento for grande?
- Neste caso, pode-se congelar apenas as primeiras camadas e treinar as seguintes
- Isso sempre ajustando a camada de saída para o seu problema



- Nas camadas treináveis, pode inicializar o gradiente descendente com os pesos pré-treinados ou descartar totalmente aquelas camadas e usar outras
- Em geral, quanto mais dado para treinar, menos camadas congeladas!

Transfer Learning - Congelamento Parcial

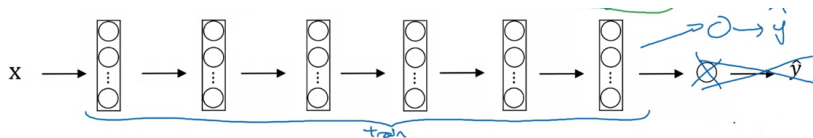
- E se o conjunto de treinamento for grande?
- Neste caso, pode-se congelar apenas as primeiras camadas e treinar as seguintes
- Isso sempre ajustando a camada de saída para o seu problema



- Nas camadas treináveis, pode inicializar o gradiente descendente com os pesos pré-treinados ou descartar totalmente aquelas camadas e usar outras
- Em geral, quanto mais dado para treinar, menos camadas congeladas!

Transfer Learning - Retreinamento Total

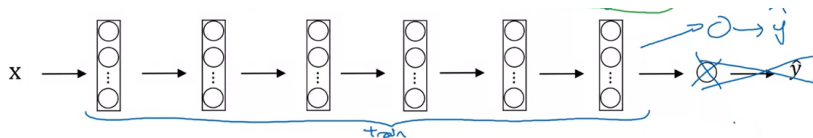
- Por fim, se tiver muitos dados, pode inicializar com os pesos pré-treinados, mas retrainar a rede toda



- NOTA: De todas as áreas do *deep learning*, a visão computacional é a que mais se beneficia do *transfer learning*
- Só não precisa ser considerado se realmente tiver muito dado e poder computacional disponível

Transfer Learning - Retreinamento Total

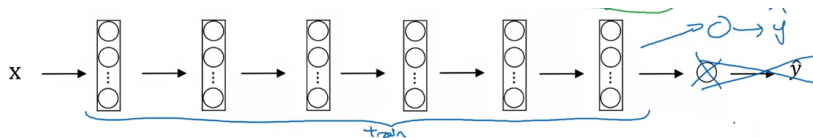
- Por fim, se tiver muitos dados, pode inicializar com os pesos pré-treinados, mas retrainar a rede toda



- NOTA: De todas as áreas do *deep learning*, a visão computacional é a que mais se beneficia do *transfer learning*
- Só não precisa ser considerado se realmente tiver muito dado e poder computacional disponível

Transfer Learning - Retreinamento Total

- Por fim, se tiver muitos dados, pode inicializar com os pesos pré-treinados, mas retrainar a rede toda



- NOTA: De todas as áreas do *deep learning*, a visão computacional é a que mais se beneficia do *transfer learning*
- Só não precisa ser considerado se realmente tiver muito dado e poder computacional disponível

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation**
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

Data Augmentation

- Problemas de visão computacional são geralmente complicados, o que exige mais dados para treinar
- *Data augmentation* desempenha papel importante neste contexto
- Isso tanto no treino com *transfer learning* ou *from scratch*

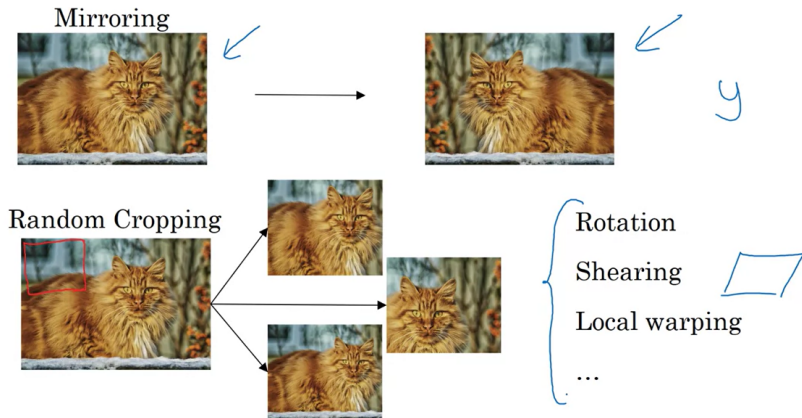
Data Augmentation

- Problemas de visão computacional são geralmente complicados, o que exige mais dados para treinar
- *Data augmentation* desempenha papel importante neste contexto
- Isso tanto no treino com *transfer learning* ou *from scratch*

Data Augmentation

- Problemas de visão computacional são geralmente complicados, o que exige mais dados para treinar
- *Data augmentation* desempenha papel importante neste contexto
- Isso tanto no treino com *transfer learning* ou *from scratch*

Data Augmentation - Métodos



Data Augmentation - Métodos

- O mais simples é o espelhamento no eixo vertical
- Outra técnica comum é o *random crop*
- NOTA: Um *crop* pode sair bem diferente da imagem original, porém se o conjunto de *crops* for grande, funciona bem
- Outras formas são menos usadas devido à complexidade, como rotação, *shearing*, deformações locais, etc.

Data Augmentation - Métodos

- O mais simples é o espelhamento no eixo vertical
- Outra técnica comum é o *random crop*
- NOTA: Um *crop* pode sair bem diferente da imagem original, porém se o conjunto de *crops* for grande, funciona bem
- Outras formas são menos usadas devido à complexidade, como rotação, *shearing*, deformações locais, etc.

Data Augmentation - Métodos

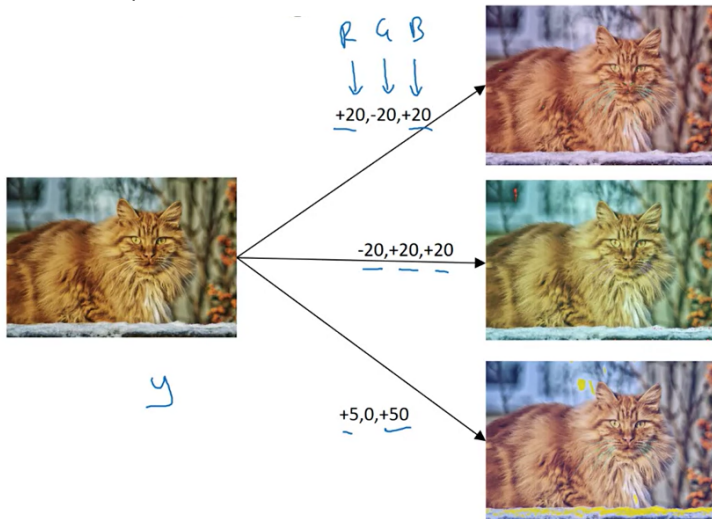
- O mais simples é o espelhamento no eixo vertical
- Outra técnica comum é o *random crop*
- NOTA: Um *crop* pode sair bem diferente da imagem original, porém se o conjunto de *crops* for grande, funciona bem
- Outras formas são menos usadas devido à complexidade, como rotação, *shearing*, deformações locais, etc.

Data Augmentation - Métodos

- O mais simples é o espelhamento no eixo vertical
- Outra técnica comum é o *random crop*
- NOTA: Um *crop* pode sair bem diferente da imagem original, porém se o conjunto de *crops* for grande, funciona bem
- Outras formas são menos usadas devido à complexidade, como rotação, *shearing*, deformações locais, etc.

Data Augmentation - Color Shifting

- Introduzir distorções diferentes em cada canal R, G e B



Data Augmentation - Color Shifting

- A 1ª imagem é tornada mais púrpura, a 2ª mais amarelada, a 3ª mais azul
- Esses porém são casos exagerados; Na prática, a distorção é obtida de uma distribuição e tem valores pequenos
- Ideia é simular alguma mudança na iluminação do sol ou artificial, tornando o modelo mais robusto a essas mudanças
- Uma técnica usada na escolha dessa distorção é baseada em PCA ("PCA color augmentation"), descrita no artigo da AlexNet
- Ideia básica é distorcer mais as cores que estão mais presentes

Data Augmentation - Color Shifting

- A 1ª imagem é tornada mais púrpura, a 2ª mais amarelada, a 3ª mais azul
- Esses porém são casos exagerados; Na prática, a distorção é obtida de uma distribuição e tem valores pequenos
- Ideia é simular alguma mudança na iluminação do sol ou artificial, tornando o modelo mais robusto a essas mudanças
- Uma técnica usada na escolha dessa distorção é baseada em PCA ("PCA color augmentation"), descrita no artigo da AlexNet
- Ideia básica é distorcer mais as cores que estão mais presentes

Data Augmentation - Color Shifting

- A 1ª imagem é tornada mais púrpura, a 2ª mais amarelada, a 3ª mais azul
- Esses porém são casos exagerados; Na prática, a distorção é obtida de uma distribuição e tem valores pequenos
- Ideia é simular alguma mudança na iluminação do sol ou artificial, tornando o modelo mais robusto a essas mudanças
- Uma técnica usada na escolha dessa distorção é baseada em PCA ("PCA color augmentation"), descrita no artigo da AlexNet
- Ideia básica é distorcer mais as cores que estão mais presentes

Data Augmentation - Color Shifting

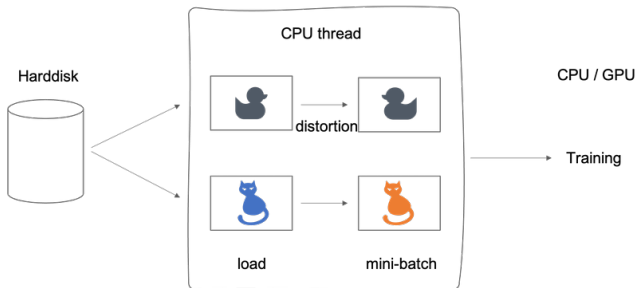
- A 1ª imagem é tornada mais púrpura, a 2ª mais amarelada, a 3ª mais azul
- Esses porém são casos exagerados; Na prática, a distorção é obtida de uma distribuição e tem valores pequenos
- Ideia é simular alguma mudança na iluminação do sol ou artificial, tornando o modelo mais robusto a essas mudanças
- Uma técnica usada na escolha dessa distorção é baseada em PCA (“PCA color augmentation”), descrita no artigo da AlexNet
- Ideia básica é distorcer mais as cores que estão mais presentes

Data Augmentation - Color Shifting

- A 1ª imagem é tornada mais púrpura, a 2ª mais amarelada, a 3ª mais azul
- Esses porém são casos exagerados; Na prática, a distorção é obtida de uma distribuição e tem valores pequenos
- Ideia é simular alguma mudança na iluminação do sol ou artificial, tornando o modelo mais robusto a essas mudanças
- Uma técnica usada na escolha dessa distorção é baseada em PCA (“PCA color augmentation”), descrita no artigo da AlexNet
- Ideia básica é distorcer mais as cores que estão mais presentes

Data Augmentation - Implementação Paralela

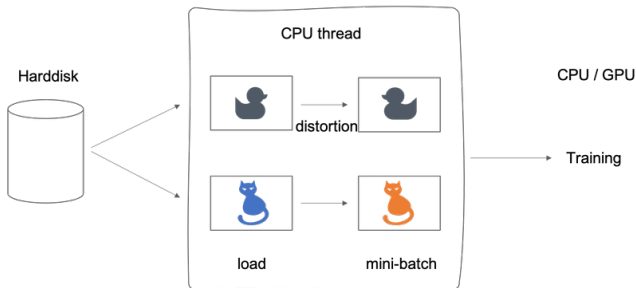
- Enquanto a CPU vai carregando os dados do disco em paralelo, ela já pode ir implementando diferentes distorções sobre essas imagens



- O *data augmentation* também tem seus hiperparâmetros (p. ex. quanto de *color shifting* aplicar)
- Novamente, partir de uma implementação *open-source* é interessante

Data Augmentation - Implementação Paralela

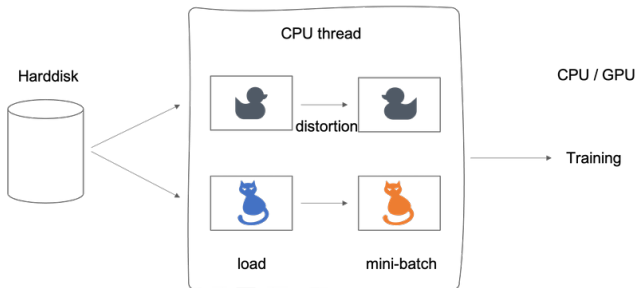
- Enquanto a CPU vai carregando os dados do disco em paralelo, ela já pode ir implementando diferentes distorções sobre essas imagens



- O *data augmentation* também tem seus hiperparâmetros (p. ex. quanto de *color shifting* aplicar)
- Novamente, partir de uma implementação *open-source* é interessante

Data Augmentation - Implementação Paralela

- Enquanto a CPU vai carregando os dados do disco em paralelo, ela já pode ir implementando diferentes distorções sobre essas imagens



- O *data augmentation* também tem seus hiperparâmetros (p. ex. quanto de *color shifting* aplicar)
- Novamente, partir de uma implementação *open-source* é interessante

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado**
- 12 Dicas para Benchmarks e Competições

Estado da Visão Computacional

- Problemas em *machine learning* podem ser categorizados de acordo com a quantidade de dados

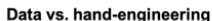
Data vs. hand-engineering



- Reconhecimento de fala apresenta uma grande quantidade de dados disponíveis
- Em reconhecimento de imagens, embora tenhamos hoje grandes conjuntos, o problema ainda é bastante complexo
- Já em problemas mais específicos, como detecção de objetos, temos dificuldade para anotar muitas *bounding boxes*

Estado da Visão Computacional

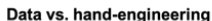
- Problemas em *machine learning* podem ser categorizados de acordo com a quantidade de dados



- Reconhecimento de fala apresenta uma grande quantidade de dados disponíveis
- Em reconhecimento de imagens, embora tenhamos hoje grandes conjuntos, o problema ainda é bastante complexo
- Já em problemas mais específicos, como detecção de objetos, temos dificuldade para anotar muitas *bounding boxes*

Estado da Visão Computacional

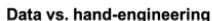
- Problemas em *machine learning* podem ser categorizados de acordo com a quantidade de dados



- Reconhecimento de fala apresenta uma grande quantidade de dados disponíveis
- Em reconhecimento de imagens, embora tenhamos hoje grandes conjuntos, o problema ainda é bastante complexo
- Já em problemas mais específicos, como detecção de objetos, temos dificuldade para anotar muitas *bounding boxes*

Estado da Visão Computacional

- Problemas em *machine learning* podem ser categorizados de acordo com a quantidade de dados



- Reconhecimento de fala apresenta uma grande quantidade de dados disponíveis
- Em reconhecimento de imagens, embora tenhamos hoje grandes conjuntos, o problema ainda é bastante complexo
- Já em problemas mais específicos, como detecção de objetos, temos dificuldade para anotar muitas *bounding boxes*

Fontes de Aprendizizado

- Algoritmo de aprendizado tem duas fontes de conhecimento: dados anotados e decisões de engenharia (*features*, arquitetura e outros)
- Quando não temos o primeiro, precisamos do segundo!
- Isso explica historicamente a tendência por desenvolvimento de arquiteturas muito complexas para suprir a falta de dados
- E isso também justifica por que algoritmos de detecção de objetos são ainda mais complexos do que reconhecimento de imagens
- Felizmente, o *transfer learning* ajuda muito neste cenário de poucos dados

Fontes de Aprendizizado

- Algoritmo de aprendizado tem duas fontes de conhecimento: dados anotados e decisões de engenharia (*features*, arquitetura e outros)
- Quando não temos o primeiro, precisamos do segundo!
- Isso explica historicamente a tendência por desenvolvimento de arquiteturas muito complexas para suprir a falta de dados
- E isso também justifica por que algoritmos de detecção de objetos são ainda mais complexos do que reconhecimento de imagens
- Felizmente, o *transfer learning* ajuda muito neste cenário de poucos dados

Fontes de Aprendizizado

- Algoritmo de aprendizado tem duas fontes de conhecimento: dados anotados e decisões de engenharia (*features*, arquitetura e outros)
- Quando não temos o primeiro, precisamos do segundo!
- Isso explica historicamente a tendência por desenvolvimento de arquiteturas muito complexas para suprir a falta de dados
- E isso também justifica por que algoritmos de detecção de objetos são ainda mais complexos do que reconhecimento de imagens
- Felizmente, o *transfer learning* ajuda muito neste cenário de poucos dados

Fontes de Aprendizizado

- Algoritmo de aprendizado tem duas fontes de conhecimento: dados anotados e decisões de engenharia (*features*, arquitetura e outros)
- Quando não temos o primeiro, precisamos do segundo!
- Isso explica historicamente a tendência por desenvolvimento de arquiteturas muito complexas para suprir a falta de dados
- E isso também justifica por que algoritmos de detecção de objetos são ainda mais complexos do que reconhecimento de imagens
- Felizmente, o *transfer learning* ajuda muito neste cenário de poucos dados

Fontes de Aprendizizado

- Algoritmo de aprendizado tem duas fontes de conhecimento: dados anotados e decisões de engenharia (*features*, arquitetura e outros)
- Quando não temos o primeiro, precisamos do segundo!
- Isso explica historicamente a tendência por desenvolvimento de arquiteturas muito complexas para suprir a falta de dados
- E isso também justifica por que algoritmos de detecção de objetos são ainda mais complexos do que reconhecimento de imagens
- Felizmente, o *transfer learning* ajuda muito neste cenário de poucos dados

Outline

- 1 Por que Olhar para Estudos de Caso
- 2 Redes Clássicas
- 3 ResNet
- 4 Network-in-Network e Convolução 1x1
- 5 Rede Inception
- 6 Depth-wise Separable Convolutions e MobileNet
- 7 Arquitetura MobileNet
- 8 EfficientNet
- 9 Dicas Práticas para o Uso de ConvNets - Open Source e Transfer Learning
- 10 Dicas Práticas - Data Augmentation
- 11 Estado Geral da Área e Fontes de Aprendizado
- 12 Dicas para Benchmarks e Competições

Dicas para Benchmarks/Competições

- Uma forma mais fácil de publicar na área é ir bem nos *benchmarks* clássicos
- Uma 1ª dica para isso é usar *ensembling*: média de várias redes (ex. 3 a 15)
- Permite melhora em *benchmarks*, mas pouco útil na prática devido ao custo computacional
- Outra opção é o *multi-crop* no teste, que é um *data augmentation* no teste

Dicas para Benchmarks/Competições

- Uma forma mais fácil de publicar na área é ir bem nos *benchmarks* clássicos
- Uma 1ª dica para isso é usar *ensembling*: média de várias redes (ex. 3 a 15)
- Permite melhora em *benchmarks*, mas pouco útil na prática devido ao custo computacional
- Outra opção é o *multi-crop* no teste, que é um *data augmentation* no teste

Dicas para Benchmarks/Competições

- Uma forma mais fácil de publicar na área é ir bem nos *benchmarks* clássicos
- Uma 1ª dica para isso é usar *ensembling*: média de várias redes (ex. 3 a 15)
- Permite melhora em *benchmarks*, mas pouco útil na prática devido ao custo computacional
- Outra opção é o *multi-crop* no teste, que é um *data augmentation* no teste

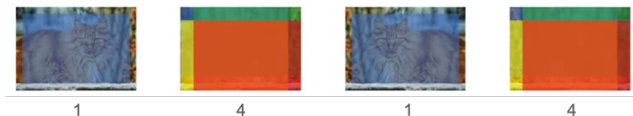
Dicas para Benchmarks/Competições

- Uma forma mais fácil de publicar na área é ir bem nos *benchmarks* clássicos
- Uma 1ª dica para isso é usar *ensembling*: média de várias redes (ex. 3 a 15)
- Permite melhora em *benchmarks*, mas pouco útil na prática devido ao custo computacional
- Outra opção é o *multi-crop* no teste, que é um *data augmentation* no teste

Dicas para Benchmarks/Competições

- Um exemplo é o *10-crop*, em que o classificador é rodado sobre um *crop* central e mais 4 nos cantos da imagem original e depois também da espelhada, totalizando 10 *crops*; No final faz a média do resultado

Multi-crop

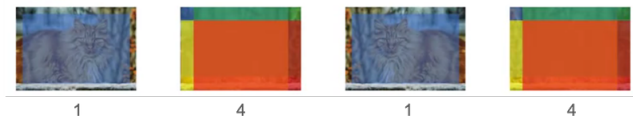


- Pode ser adaptado para um uso mais prático, se tiver hardware suficiente ou reduzir o número de *crops*
- Uma vantagem do *multi-crop* em relação ao *ensembling* é que não precisa manter todas as redes em memória

Dicas para Benchmarks/Competições

- Um exemplo é o *10-crop*, em que o classificador é rodado sobre um *crop* central e mais 4 nos cantos da imagem original e depois também da espelhada, totalizando 10 *crops*; No final faz a média do resultado

Multi-crop

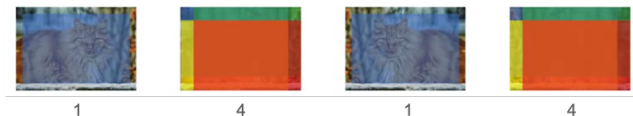


- Pode ser adaptado para um uso mais prático, se tiver hardware suficiente ou reduzir o número de *crops*
- Uma vantagem do *multi-crop* em relação ao *ensembling* é que não precisa manter todas as redes em memória

Dicas para Benchmarks/Competições

- Um exemplo é o *10-crop*, em que o classificador é rodado sobre um *crop* central e mais 4 nos cantos da imagem original e depois também da espelhada, totalizando 10 *crops*; No final faz a média do resultado

Multi-crop



- Pode ser adaptado para um uso mais prático, se tiver hardware suficiente ou reduzir o número de *crops*
- Uma vantagem do *multi-crop* em relação ao *ensembling* é que não precisa manter todas as redes em memória

Dicas para Benchmarks/Competições

- Implementações prontas ajudam muito para poucos dados pois uma arquitetura complexa que funciona em um problema de visão costuma também funcionar em outro (até surpreendentemente!)
- Usar modelos pré-treinados e fazer apenas o *fine-tuning* no dado específico também ajuda muito

Dicas para Benchmarks/Competições

- Implementações prontas ajudam muito para poucos dados pois uma arquitetura complexa que funciona em um problema de visão costuma também funcionar em outro (até surpreendentemente!)
- Usar modelos pré-treinados e fazer apenas o *fine-tuning* no dado específico também ajuda muito